

Chapter 2 Introduction to ANOVA and Linear Regression

```
## C:\Users\sjsimmo2\AppData\Local\R\win-library\4.3\reticulate\python\rpytools\  
##   return _find_and_load(name, import_)  
## C:\Users\sjsimmo2\AppData\Local\R\win-library\4.3\reticulate\python\rpytools\  
##   return _find_and_load(name, import_)
```

This Chapter aims to answer the following questions:

- What is a predictive model versus an explanatory model?
- How to perform an honest assessment of a model.
- How to estimate associations.
 - Continuous-Continuous
 - Continuous-Categorical
 - Pearson's correlation
 - Test of Hypothesis
 - Effect of outliers
 - Correlation Matrix
- How to perform ANOVA.
 - Testing assumptions
 - Kruskal-Wallis

- Post-hoc tests
- How to perform Simple Linear Regression.
 - Assumptions
 - Inference

In this chapter, we introduce one of the most commonly used tools in data science: the linear model. We'll start with some basic terminology. A **linear model** is an equation that typically takes the form

$$\mathbf{y} = \beta_0 + \beta_1 \mathbf{x}_1 + \cdots + \beta_k \mathbf{x}_k + \epsilon \quad (2.1)$$

The left-hand side of this equation, $\mathbf{y}$ is equivalently called the **target**, **response**, or **dependent** variable. The right-hand side is a linear combination of the columns $\{\mathbf{x}_i\}_{i=1}^k$ which are commonly referred to as **explanatory**, **input**, **predictor**, or **independent** variables.

2.1 Predictive vs. Explanatory

The purpose of a linear model like Equation (2.1) is generally two-fold:

1. The model is **predictive** in that it can estimate the value of y for a given combination of the x attributes.
2. The model is **explanatory** in that it can estimate how y changes for a unit increase in a given x attribute, holding all else constant (via the slope parameters β).

However, it's common for *one* of these purposes to be more aligned with the specific goals of your project, and it is common to approach the building of such a model differently for each purpose.

In predictive modeling, you are most interested in how much error your model has on *holdout data*, that is, validation or test data. This is a notion that we introduce next in Section 2.2. If good predictions are all you want from your model, you are unlikely to care how many variables (including polynomial and interaction terms) are included in the final model.

In explanatory modeling, you foremost want a model that is simple to interpret and doesn't have too many input variables. It's common to avoid many polynomial and interaction terms for explanatory models. While the error rates on holdout data will still be useful reporting metrics for explanatory models, it will be more important to craft the model for ease of interpretation.

2.2 Honest Assessment

When performing predictive or explanatory modeling, we *always* divide our data into subsets for training, validation, and/or final testing. Because models are prone to discovering small, spurious patterns on the data that is used to create them (the *training data*), we set aside the *validation* and *testing data* to get a clear view of how they might perform on new data that they've never seen before. This is a concept that will be revisited several times throughout this text, highlighting its importance to honest assessment of models.

There is no single right answer for how this division should occur for every data set - the answer depends on a multitude of factors that are beyond the scope of our present discussion. Generally speaking, one expects to keep about 70% of the data for model training purposes, and the remaining 30% for validation and testing. These proportions may change depending on the amount of data available. If one has millions of observations, they can often get away with a much smaller proportion of training data to reduce computation time and increase confidence in validation. If one has substantially fewer observations, it may be necessary to increase the training proportion in order to build a sound model - trading validation confidence for proper training.

Below we demonstrate one technique for separating the data into just two subsets: training and test. These two subsets will suffice for our analyses in this text. We'll use 70% of our data for the training set and the remainder for testing.

Since we are taking a random sample, each time you run this functions you will get a different result. This can be difficult for team members who wish to keep their analyses in sync. To avoid that variation of results, we can provide a "seed" to the internal random number

generation process, which ensures that the randomly generated output is the same to all who use that seed.

The following code demonstrates sampling via the `tidyverse`. This method requires the use of an id variable. If your data set has a unique identifier built in, you may omit the first line of code (after `set.seed()`) and use that unique identifier in the third line.

```
library(tidyverse)

set.seed(123)

ames <- ames %>% mutate(id = row_number())

train <- ames %>% sample_frac(0.7)

test <- anti_join(ames, train, by = 'id')

dim(train)

## [1] 2051 82

dim(test)

## [1] 879 82
```

2.2.1 Python Code

To create the training data set in Python

```

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

train, test = train_test_split(ames_py, test_size=0.3, random_state=123)
train.head()

##                MS_SubClass  ...  Latitude
## 2278  One_Story_1946_and_Newer_All_Styles  ...  41.992318
## 1379  One_Story_1946_and_Newer_All_Styles  ...  42.031571
## 2182    PUD_Multilevel_Split_Level_Foyer  ...  42.018973
## 1436    Two_Story_1946_and_Newer  ...  42.017423
## 1599    Two_Story_PUD_1946_and_Newer  ...  41.992133
##
## [5 rows x 81 columns]

```

However, note that this will create a different train/test split. Therefore, we will just pull in our train/test data set from R.

```

train = r.train
test = r.test

```

2.3 Bivariate EDA

As stated in Chapter 1, exploratory data analysis is the foundation of any successful data science project. As we move on to the discussion of modeling, we begin to explore *bivariate* relationships in our data. In doing so, we will often explore the input variables' relationships with the target. **Such exploration should only be done on the training data; we should never let insights from the validation or test data inform our decisions about modeling.**

Bivariate exploratory analysis is often used to assess relationships between two variables. An **association** or **relationship** exists when the expected value of one variable changes at different levels of the other variable. A **linear relationship** between two continuous variables can be inferred when the general shape of a scatter plot of the two variables is a straight line.

2.3.1 Continuous-Continuous Associations

Let's conduct a preliminary assessment of the relationship between the size of the house in square feet (via `Gr_Liv_Area`) and the `Sale_Price` by creating a scatter plot (only on the training data). Note that we call this a preliminary assessment because we should not declare a statistical relationship without a formal hypothesis test (see Section 2.6).

```
ggplot(data = train) +  
  geom_point(mapping = aes(x = Gr_Liv_Area, y = Sale_Price/1000)) +  
  labs(y = "Sales Price (Thousands $)", x = "Greater Living Area (Sqft)")
```



Figure 2.1: Scatter plot demonstrating a positive linear relationship

2.3.2 Continuous-Categorical Associations

We'll also revisit the plots that we created in Section 1.1, this time being careful to use only our training data since our goal is eventually to use a linear model to predict `Sale_Price` .

We start by exploring the relationship between the external quality rating of the home (via the ordinal variable `Exter_Qual` and the `Sale_Price`).

The simplest graphic we may wish to create is a bar chart like Figure 2.2 that shows the average sale price of homes with each value of exterior quality.

```
ggplot(train) +  
  geom_bar(aes(x=Exter_Qual,y= Sale_Price),  
           position = "dodge", stat = "summary", fun = "mean") +  
  scale_y_continuous(labels = function(x) format(x, scientific = FALSE)) # Modify
```

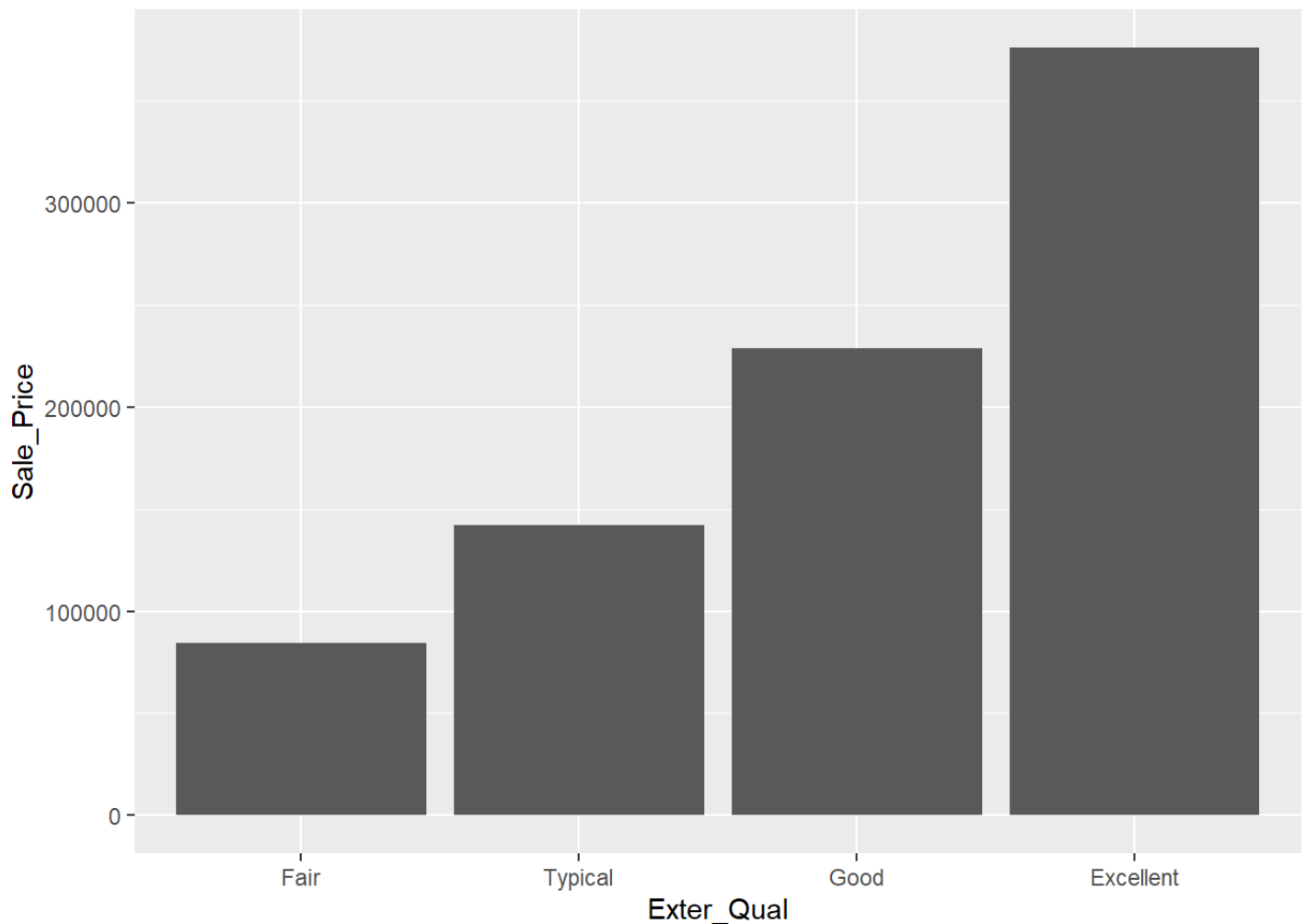


Figure 2.2: Bar Chart Comparing Average Sale Price of Homes with each Level of Exterior Quality

This gives us the idea that there may be an association between these two attributes, but it can be tricky to rely solely on this graph without exploring the overall distribution in sale price for each group. While this chart is great for the purposes of *reporting* (once we've verified the relationship), it's not the best one for exploratory analysis. The next two charts allow us to have much more information on one graphic.

The frequency histogram in Figure 2.3 allows us to see that much fewer of the homes have a rating of `Excellent` versus the other tiers, a fact that makes it difficult to compare the distributions. To normalize that quantity and compare the raw probability densities, we can change our axes to density (which is analogous to percentage) and employ a kernel density estimator with the `geom_density` plot as shown in Figure 2.4. We can then clearly see that as the exterior quality of the home “goes up” (in the ordinal sense, not in the linear sense), the sale price of the home also increases.

```
ggplot(train,aes(x=Sale_Price/1000, fill=Exter_Qual)) +  
  geom_histogram(alpha=0.2, position="identity") +  
  labs(x = "Sales Price (Thousands $)")  
  
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```

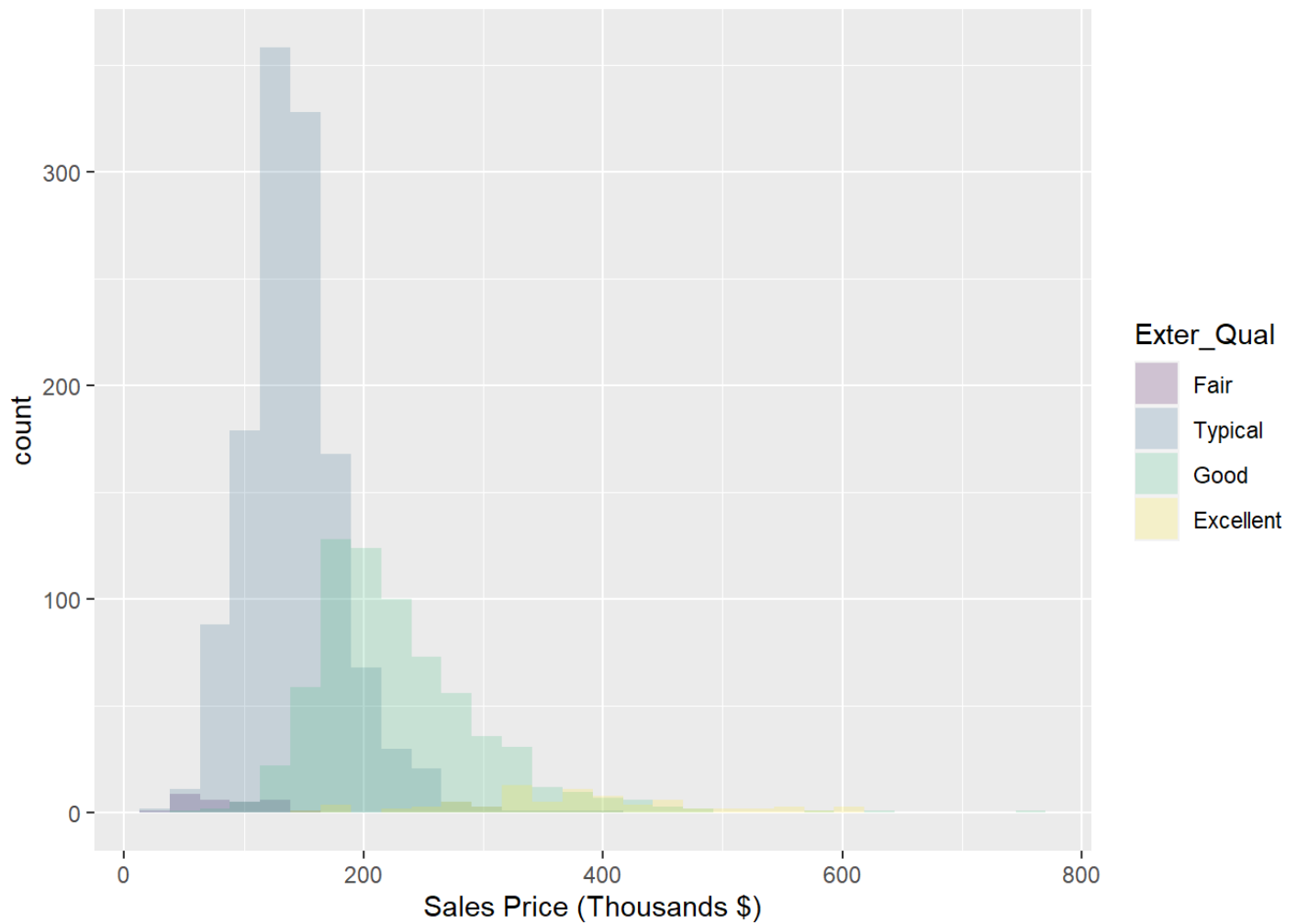


Figure 2.3: Histogram: Frequency of Sale_Price for varying qualities of home exterior

```
ggplot(ames, aes(x=Sale_Price/1000, fill=Exter_Qual)) +  
  geom_density(alpha=0.2, position="identity") +  
  labs(x = "Sales Price (Thousands $)")
```

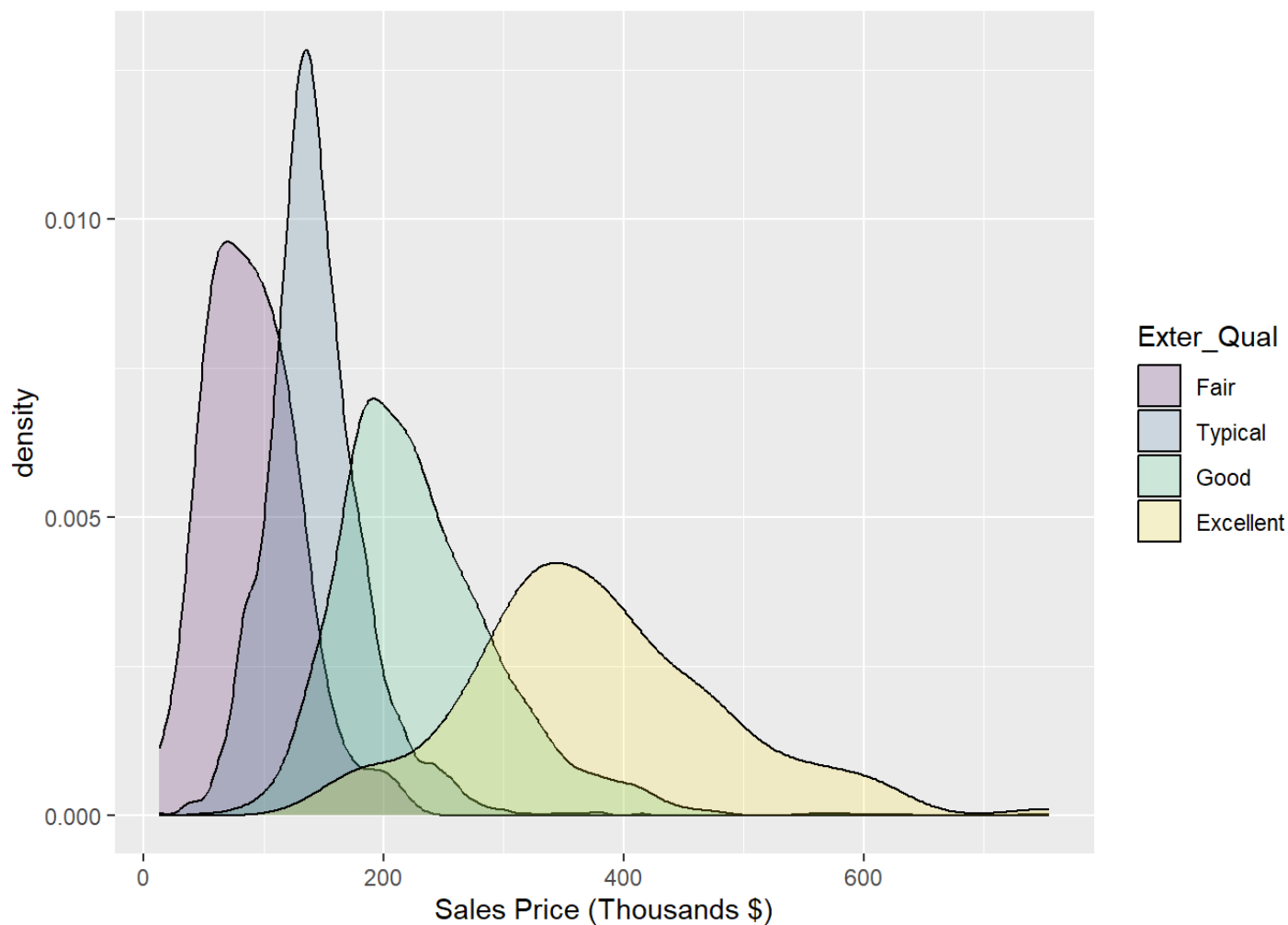


Figure 2.4: Histogram: Density of Sale_Price for varying qualities of home exterior

To further explore the location and spread of the data, we can create box-plots for each group using the following code:

```
ggplot(data = train, aes(y = Sale_Price/1000, x = `Exter_Qual`, fill = `Exter_Qual`)) +
  geom_boxplot() +
  labs(y = "Sales Price (Thousands $)", x = "Exterior Quality Category") +
  stat_summary(fun = mean, geom = "point", shape = 20, size = 5, color = "red",
    scale_fill_brewer(palette="Blues") + theme_classic() + coord_flip()
```

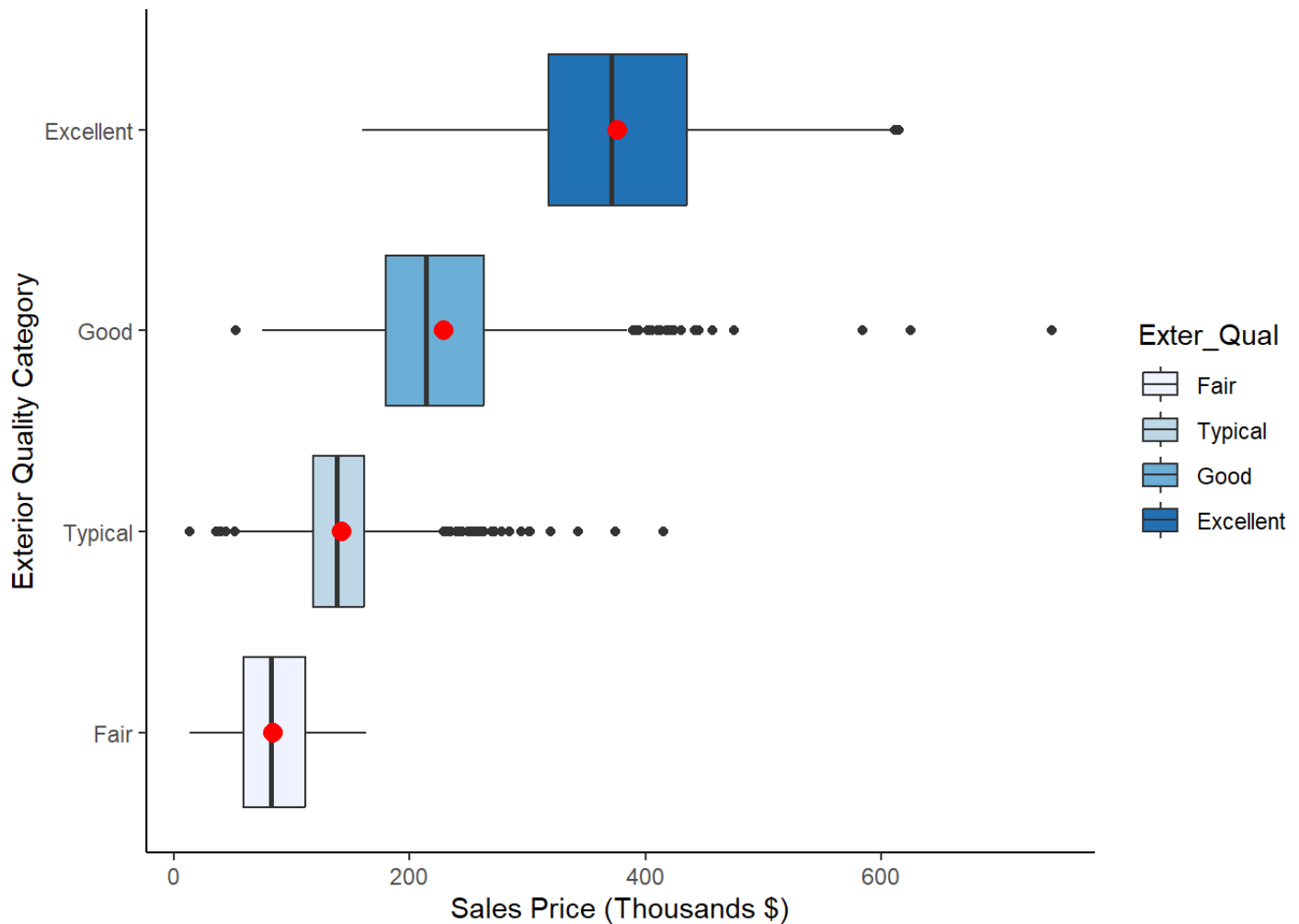


Figure 2.5: Box Plots of Sale_Price for each level of Exter_Qual

Notice that we've highlighted the mean on each box-plot for the purposes of comparison. We now have a hypothesis that we may want to formally test. After all, it is not good practice to look at Figures 2.4 and 2.5 and *declare* that a statistical difference exists. While we do, over time, get a feel for which visually apparent relationships turn out to be statistically significant, it's *imperative* that we conduct formal testing before declaring such insights to a colleague or stakeholder.

If we want to test whether the `Sale_Price` is different for the different values of `Exter_Qual`, we have to reach for the multi-group alternative to the 2-sample t-test. This is called **Analysis of Variance**, or **ANOVA** for short.

2.3.3 Python Code

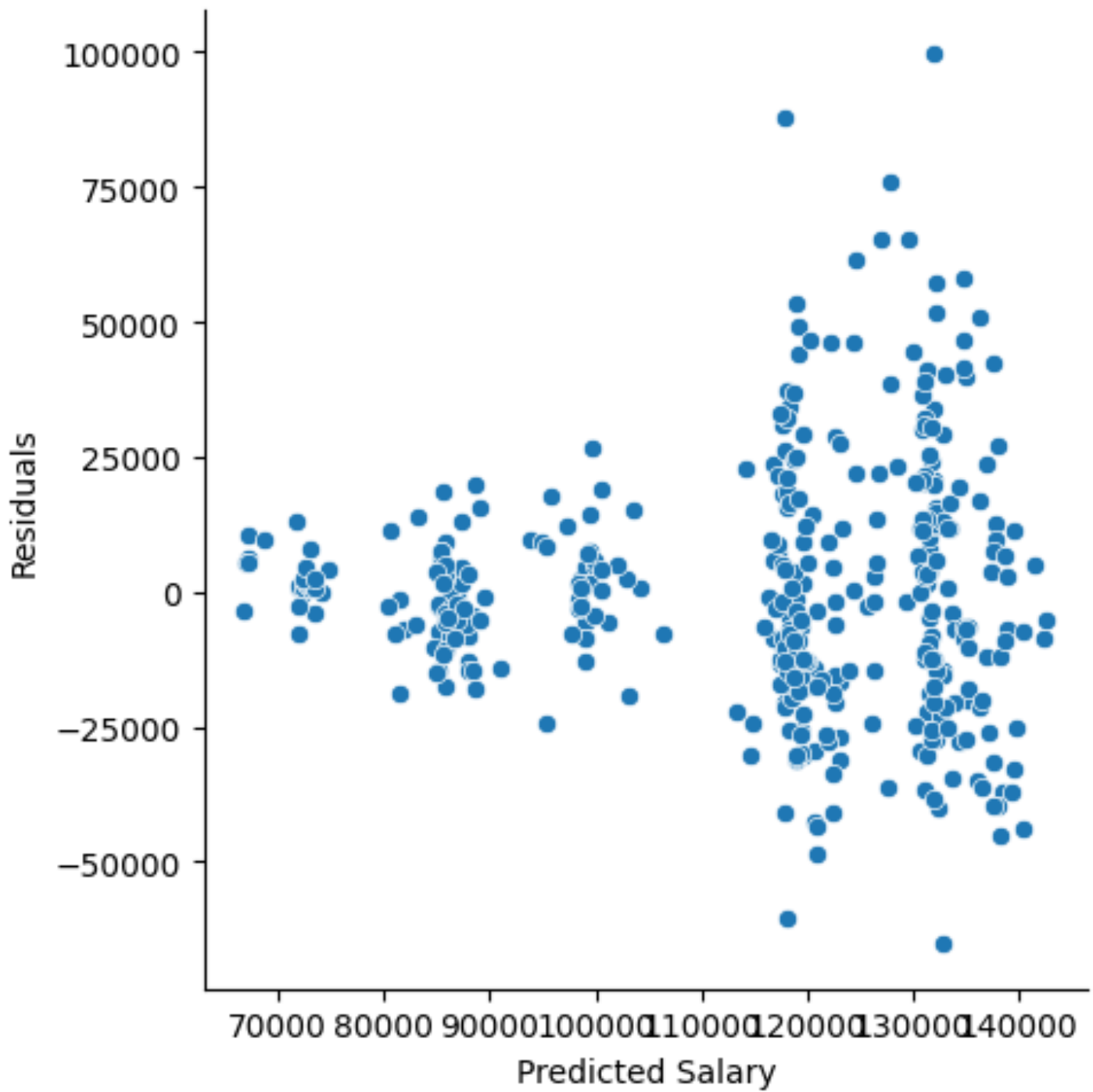
For Continuous-Continuous Associations:

```
from scipy.stats import gaussian_kde

plt.scatter(train['Gr_Liv_Area'], [price/1000 for price in train['Sale_Price']])

# Setting labels
plt.xlabel('Greater Living Area (Sqft)')
plt.ylabel('Sales Price (Thousands $)')

# Showing the plot
plt.show()
```



For Continuous-Categorical Associations:

```
# Calculate mean Sale Price for each Exterior Quality
```

```
train['Exter_Qual'] = train['Exter_Qual'].cat.remove_unused_categories()
```

```
mean_prices = train.groupby('Exter_Qual')['Sale_Price'].mean()
```

```
# Plotting the data
```

```
## <string>:1: FutureWarning: The default of observed=False is deprecated and wi
```

```
fig, ax = plt.subplots()
```

```
mean_prices.plot(kind='bar', ax=ax)
```

```
# Setting labels
```

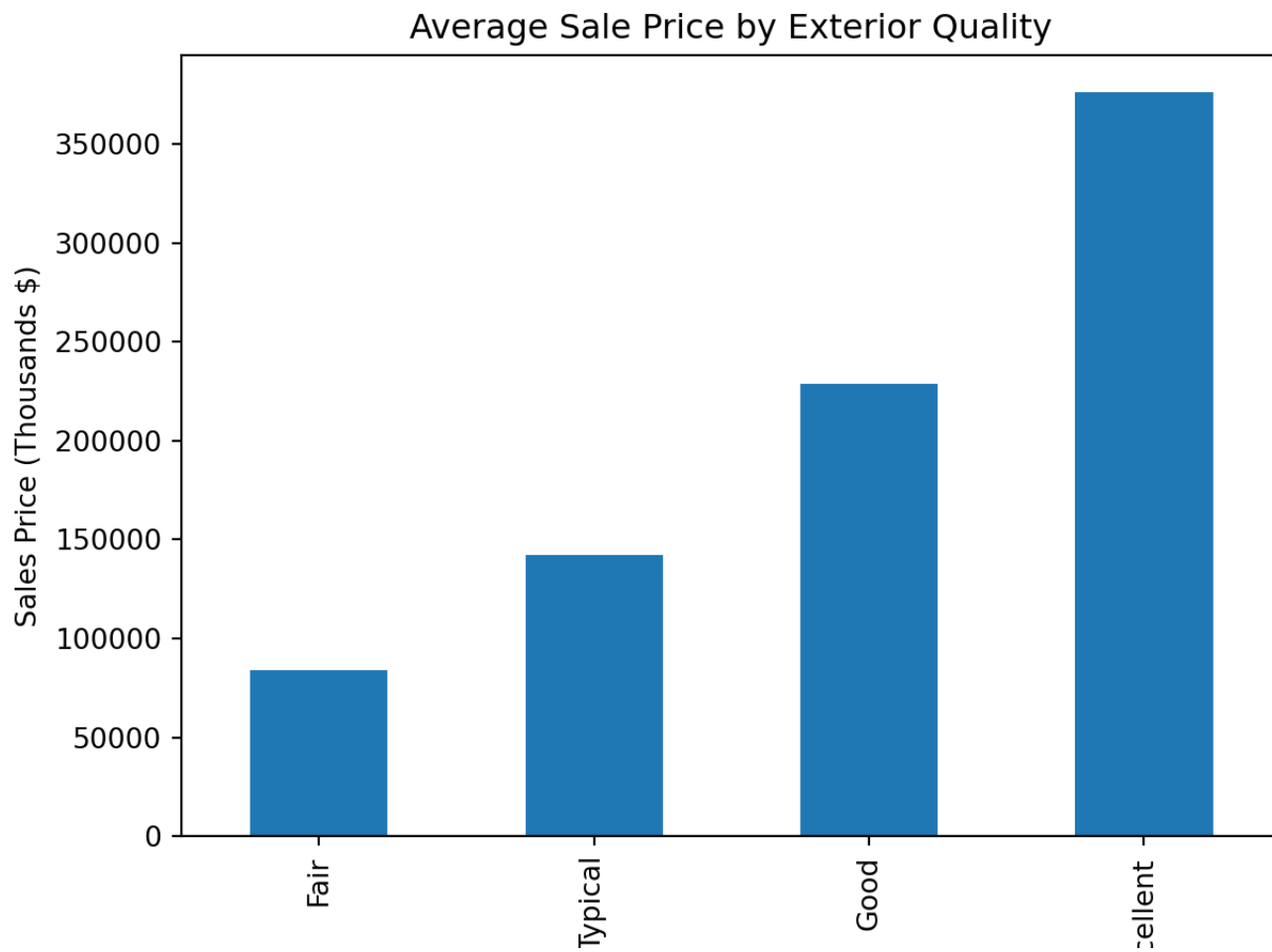
```
ax.set_ylabel('Sales Price (Thousands $)')
```

```
ax.set_xlabel('Exterior Quality')
```

```
ax.set_title('Average Sale Price by Exterior Quality')
```

```
# Displaying the plot
```

```
plt.show()
```



```

ames_py['Sale_Price2'] = ames_py['Sale_Price'] / 1000

# Get unique categories for coloring
unique_categories = ames_py['Exter_Qual'].unique()
colors = plt.cm.viridis(np.linspace(0, 1, len(unique_categories)))

# Plotting the data
fig, ax = plt.subplots()

for category, color in zip(unique_categories, colors):
    subset = ames_py[ames_py['Exter_Qual'] == category]
    ax.hist(subset['Sale_Price2'], bins=10, alpha=0.5, label=category, color=color)

# Setting labels

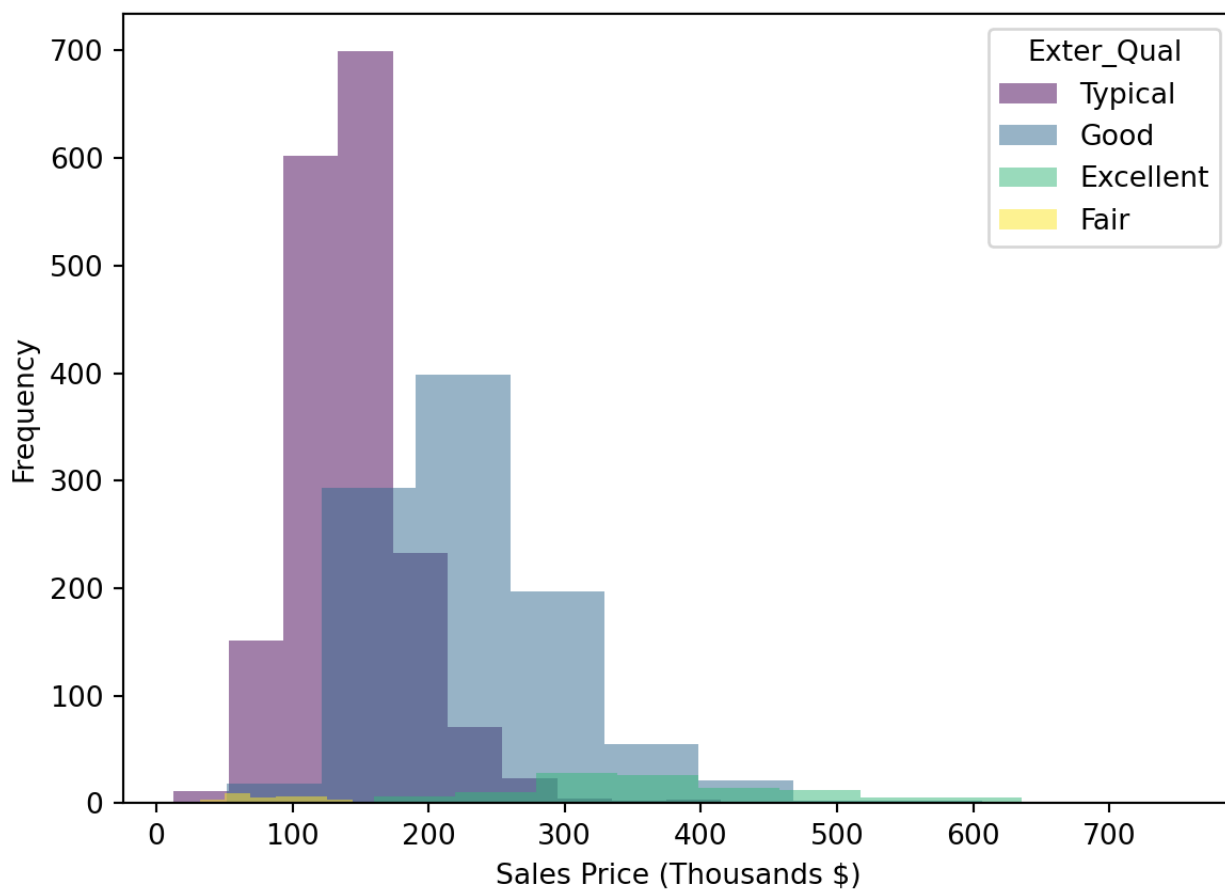
## (array([ 11., 151., 602., 699., 233., 71., 23., 4., 2., 3.]), array([
##      254.1156, 294.3367, 334.5578, 374.7789, 415.    ]), <BarContainer object of 10 artists>)
## (array([ 18., 293., 399., 197., 55., 21., 2., 2., 1., 1.]), array([
##      675.7, 745. ]), <BarContainer object of 10 artists>)
## (array([ 6., 10., 28., 26., 14., 12., 5., 5., 0., 1.]), array([160. , 219.
##      695.5, 755. ]), <BarContainer object of 10 artists>)
## (array([1., 3., 9., 5., 6., 6., 3., 0., 1., 1.]), array([ 13.1 , 31.79, 50.4
##      162.62, 181.31, 200.  ]), <BarContainer object of 10 artists>)

```

```
ax.set_xlabel('Sales Price (Thousands $)')  
ax.set_ylabel('Frequency')  
ax.legend(title='Exter_Qual')
```

```
# Displaying the plot
```

```
plt.show()
```



```
# Get unique categories for coloring
```

```
unique_categories = ames_py['Exter_Qual'].unique()  
colors = plt.cm.viridis(np.linspace(0, 1, len(unique_categories)))
```

```
# Plotting the data
```

```
fig, ax = plt.subplots()
```

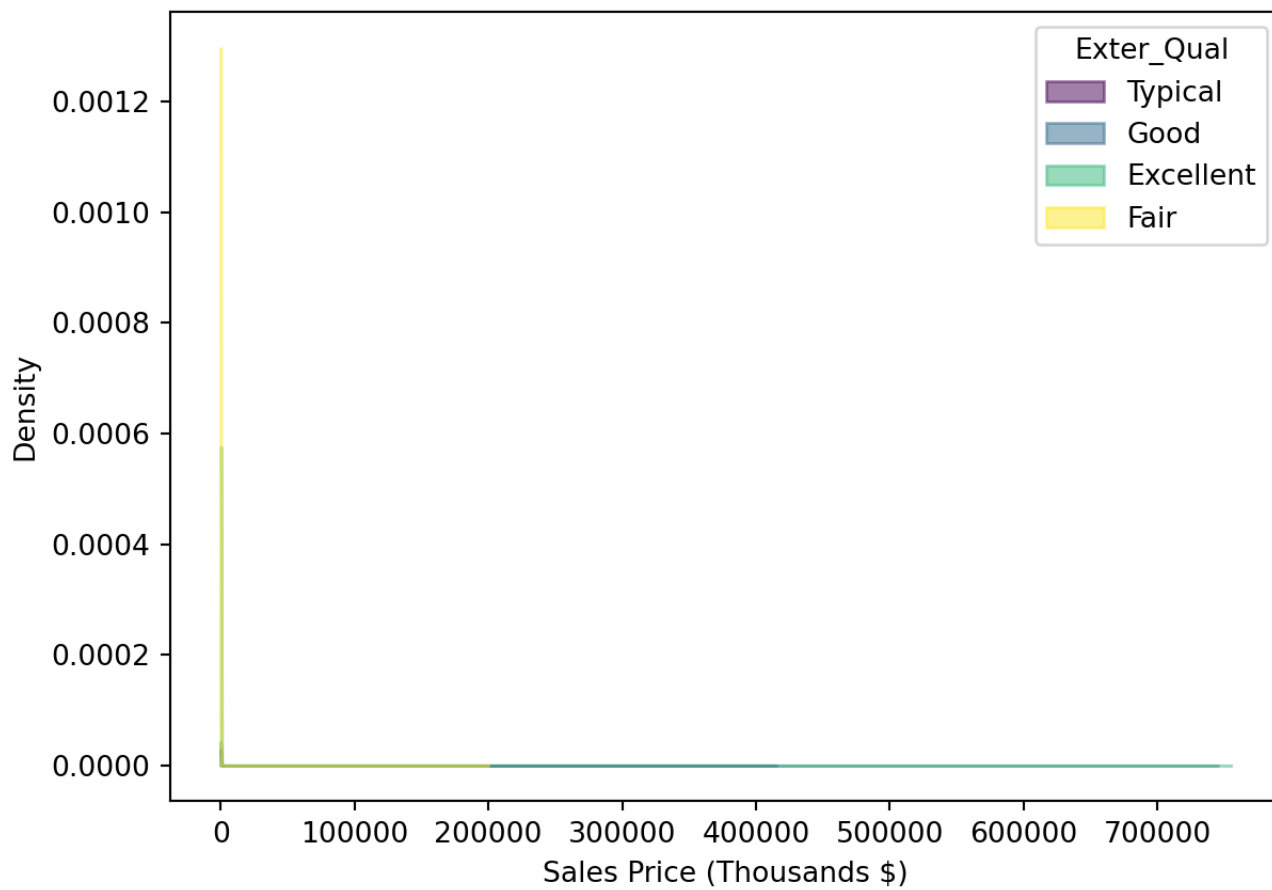
```
for category, color in zip(unique_categories, colors):  
    subset = ames_py[ames_py['Exter_Qual'] == category]  
    kde = gaussian_kde(subset['Sale_Price2'])  
    x = np.linspace(subset['Sale_Price2'].min(), subset['Sale_Price'].max(), 100)  
    y = kde(x)  
    ax.fill_between(x, y, alpha=0.5, label=category, color=color)
```

```
# Setting labels
```

```
ax.set_xlabel('Sales Price (Thousands $)')  
ax.set_ylabel('Density')  
ax.legend(title='Exter_Qual')
```

```
# Displaying the plot
```

```
plt.show()
```



```
# Plotting the data
```

```
fig, ax = plt.subplots()
```

```
# Create box plot
```

```
ames_py.boxplot(column='Sale_Price', by='Exter_Qual', ax=ax)
```

```
# Set labels
```

```
ax.set_xlabel('Exter_Qual')
```

```
ax.set_ylabel('Sale Price')
```

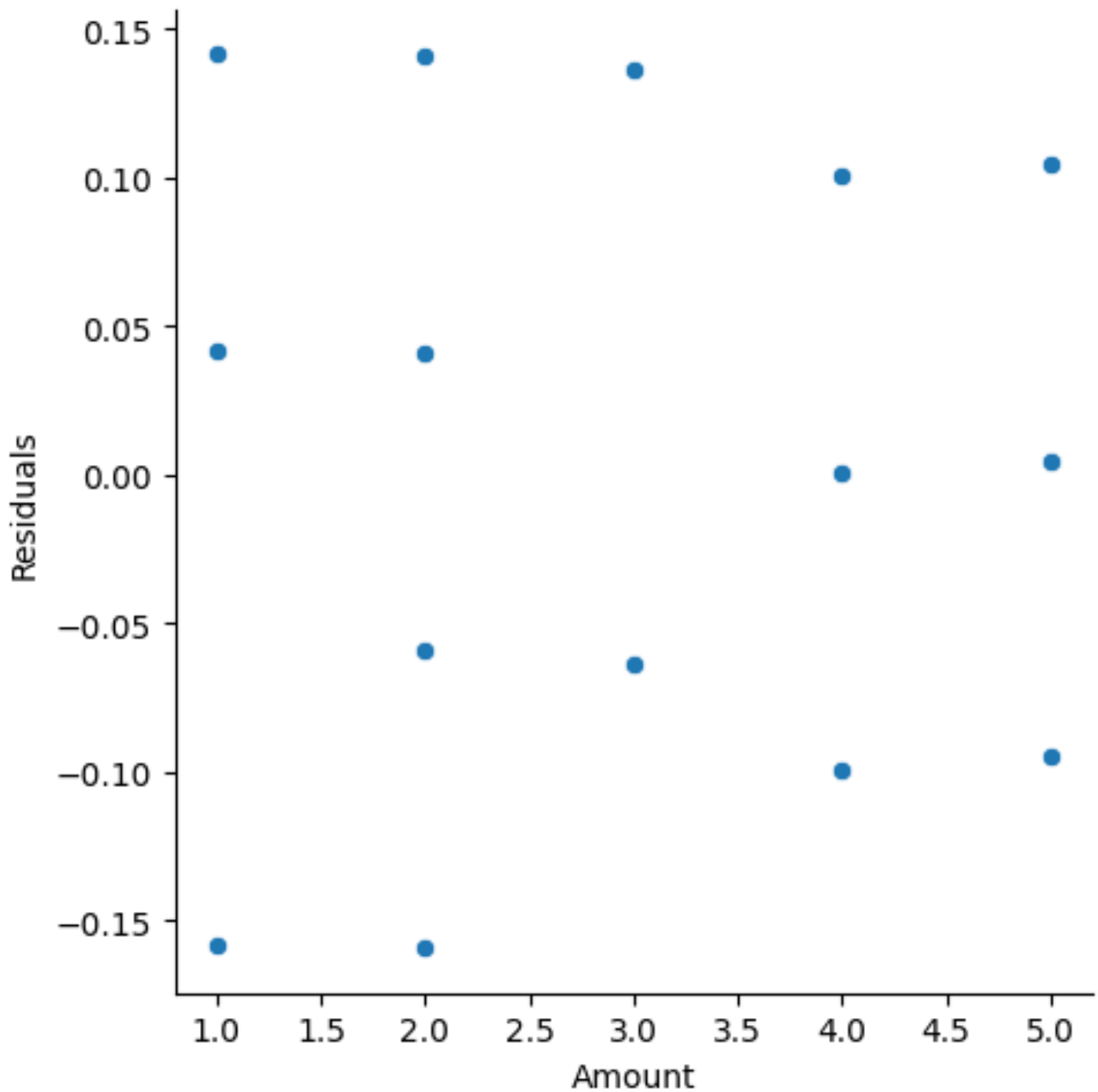
```
ax.set_title('Sale Price by Exterior Quality')
```

```
# Remove the automatic 'Boxplot grouped by Exter_Qual' title
```

```
plt.suptitle('')
```

```
# Display the plot
```

```
plt.show()
```



2.4 One-Way ANOVA

One-way ANOVA aims to determine whether there is a difference in the mean of a continuous attribute across levels of a categorical attribute. Sound like a two-sample t-test? Indeed, it's the extension of that test to more than two groups. Performing ANOVA with a binary input variable is mathematically identical to the two-sample t-test, assuming the variances are equal:

1. The observations are independent
2. The model residuals are normally distributed
3. The variances for each group are equal

A one-way ANOVA refers to a single hypothesis test, which is $H_0 : \mu_1 = \mu_2 = \dots \mu_k$ for a predictor variable with k levels against the alternative of at least one difference. We will go back to the cars data set. There is another cars data set called "cars2" that adds cars from Germany. We can visualize this information via boxplots and density plots:

```
library(ggpubr)

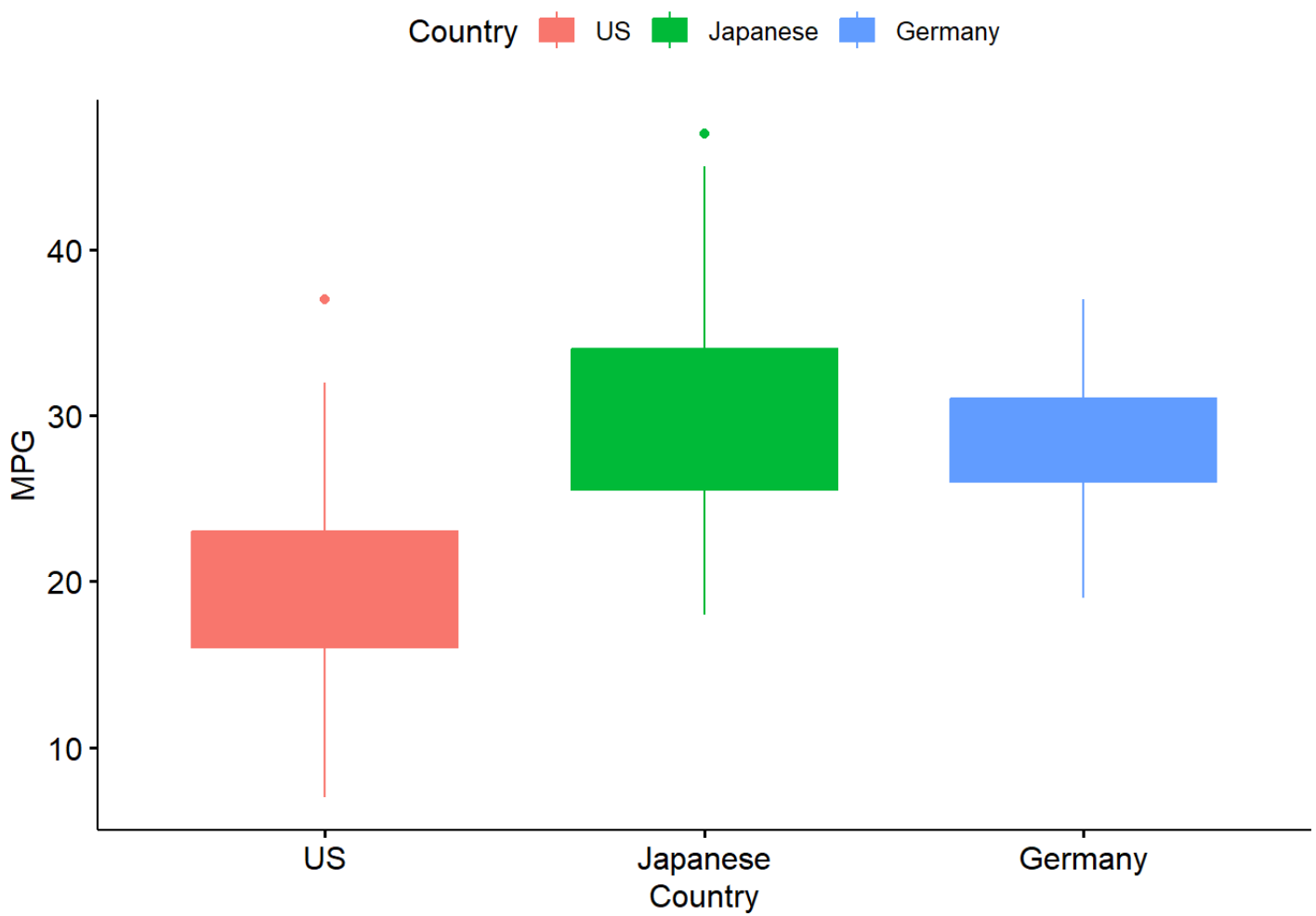
cars2<-read.csv("https://raw.githubusercontent.com/IAA-Faculty/statistical_founda

ggboxplot(cars2,x="Country",y="MPG", add="mean",color="Country",fill="Country")

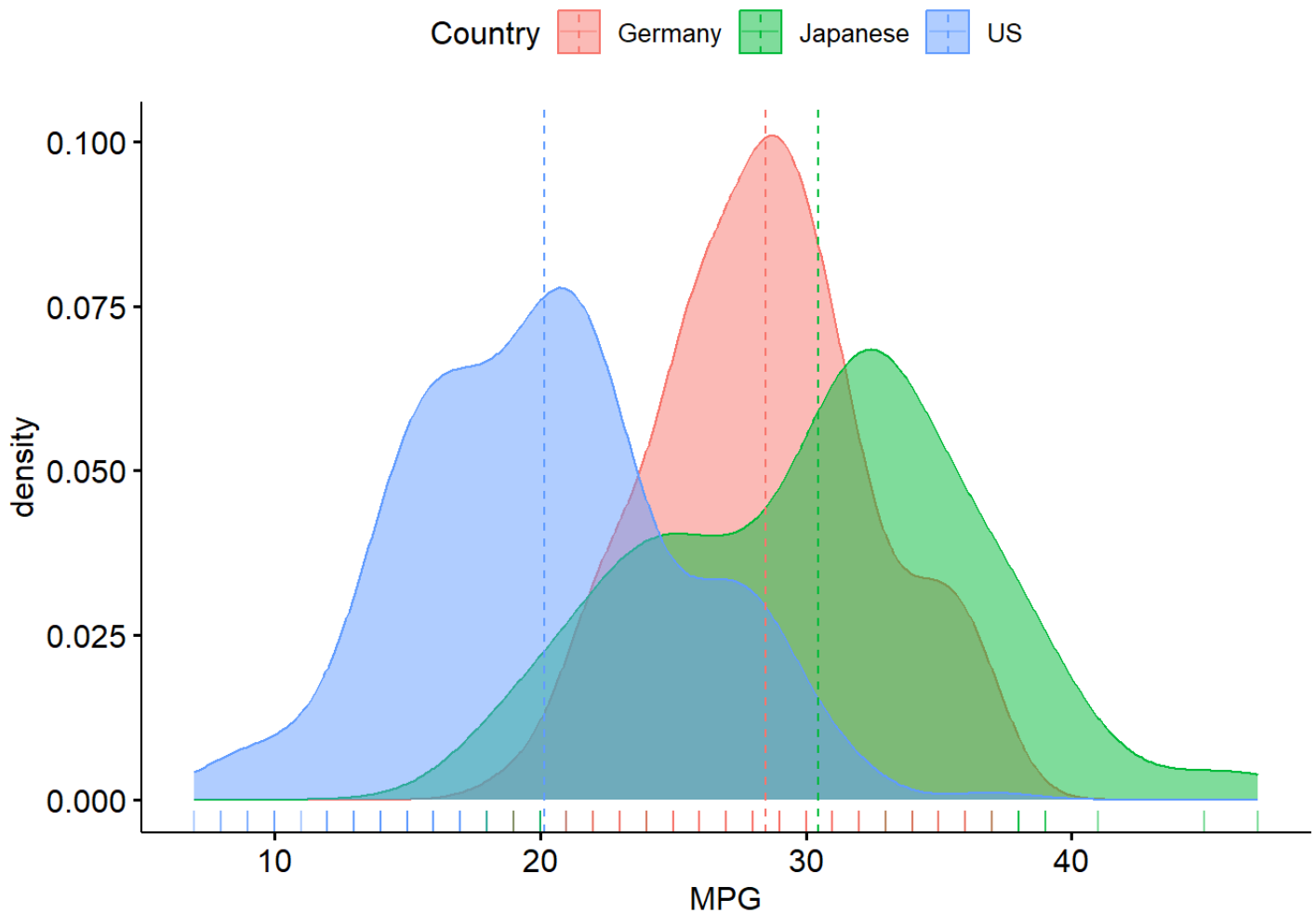
## Warning: The `fun.y` argument of `stat_summary()` is deprecated as of ggplot2
## i Please use the `fun` argument instead.
## i The deprecated feature was likely used in the ggpubr package.
## Please report the issue at <https://github.com/kassambara/ggpubr/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: The `fun.ymin` argument of `stat_summary()` is deprecated as of ggplot2 3.4.0.
## i Please use the `fun.min` argument instead.
## i The deprecated feature was likely used in the ggpubr package.
## Please report the issue at <https://github.com/kassambara/ggpubr/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
## Warning: The `fun.ymax` argument of `stat_summary()` is deprecated as of ggplot2 3.4.0.
## i Please use the `fun.max` argument instead.
## i The deprecated feature was likely used in the ggpubr package.
## Please report the issue at <https://github.com/kassambara/ggpubr/issues>.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

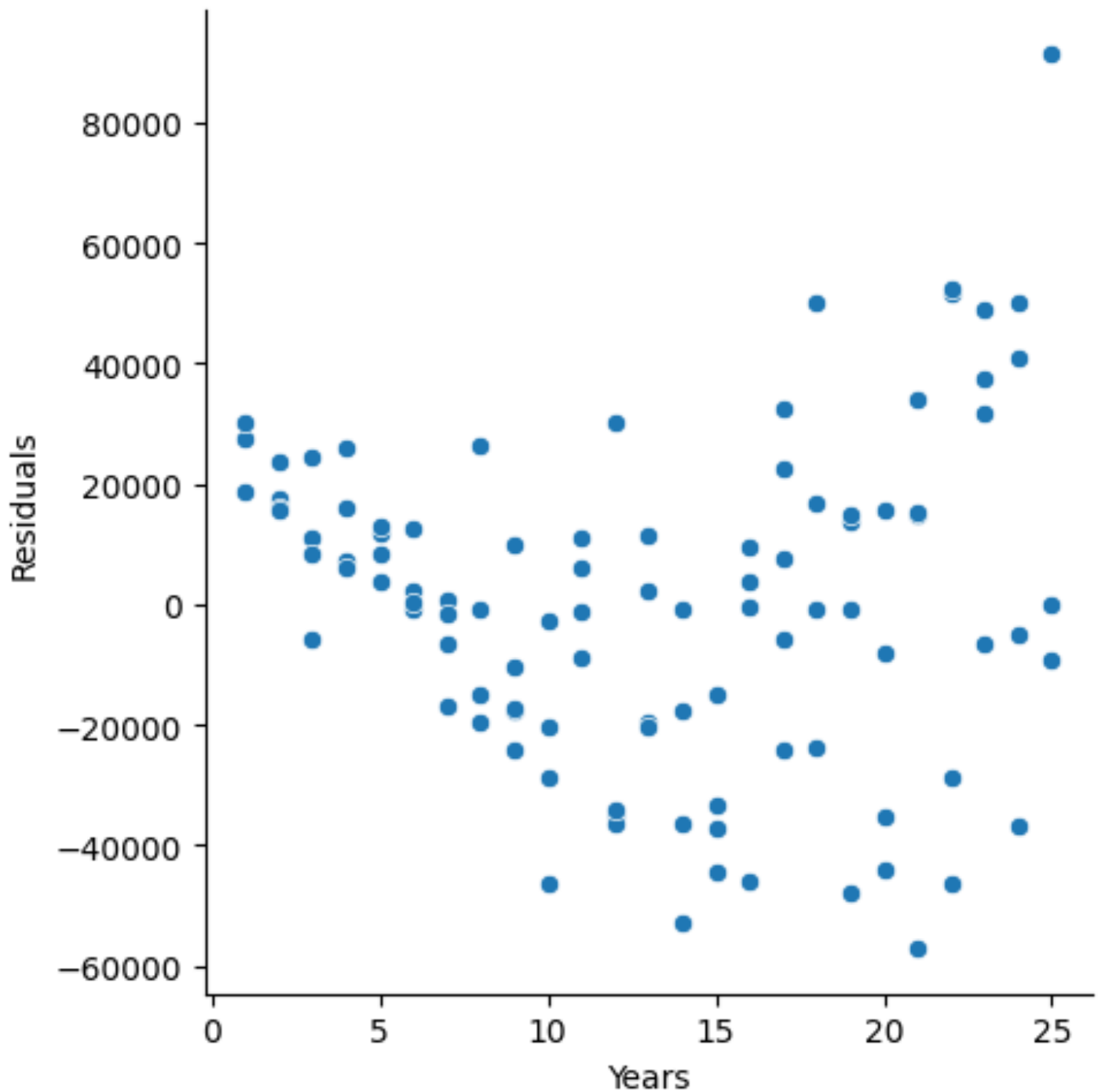


```
ggdensity(cars2, x = "MPG",  
          add = "mean", rug = TRUE,  
          color = "Country", fill = "Country")
```



To test that the means are equal, we can use the function `aov`. However, we must first assess the assumptions (normality within each group and variances are equal). We can assess the normality assumption by using a QQ-plot on the residuals:

```
model<-aov(MPG~factor(Country), data=cars2)
ggqqplot(residuals(model))
```



Based on residual plot, Normality appears to be a reasonable assumption.

We can also perform an hypothesis test for Normality on the residuals:

H_0 : *The residuals are Normally distributed*

H_A : *The residuals are not Normally distributed*

```
shapiro.test(residuals(model))

##
##  Shapiro-Wilk normality test
##
## data:  residuals(model)
## W = 0.99322, p-value = 0.05124
```

Same conclusion as QQ-plot (although, they can differ!!).

Yes, we do have to run the model first to get the residuals, but we should NOT look at the output of the model until we are comfortable that the assumptions hold. Since Normality is fine here, we move on to testing the variances are equal using the Levene test. The hypothesis test is:

$H_0 : \sigma_a^2 = \sigma_b^2 = \sigma_c^2 = \sigma_d^2$ i.e., the groups have equal variance

H_a : at least one group's variance is different

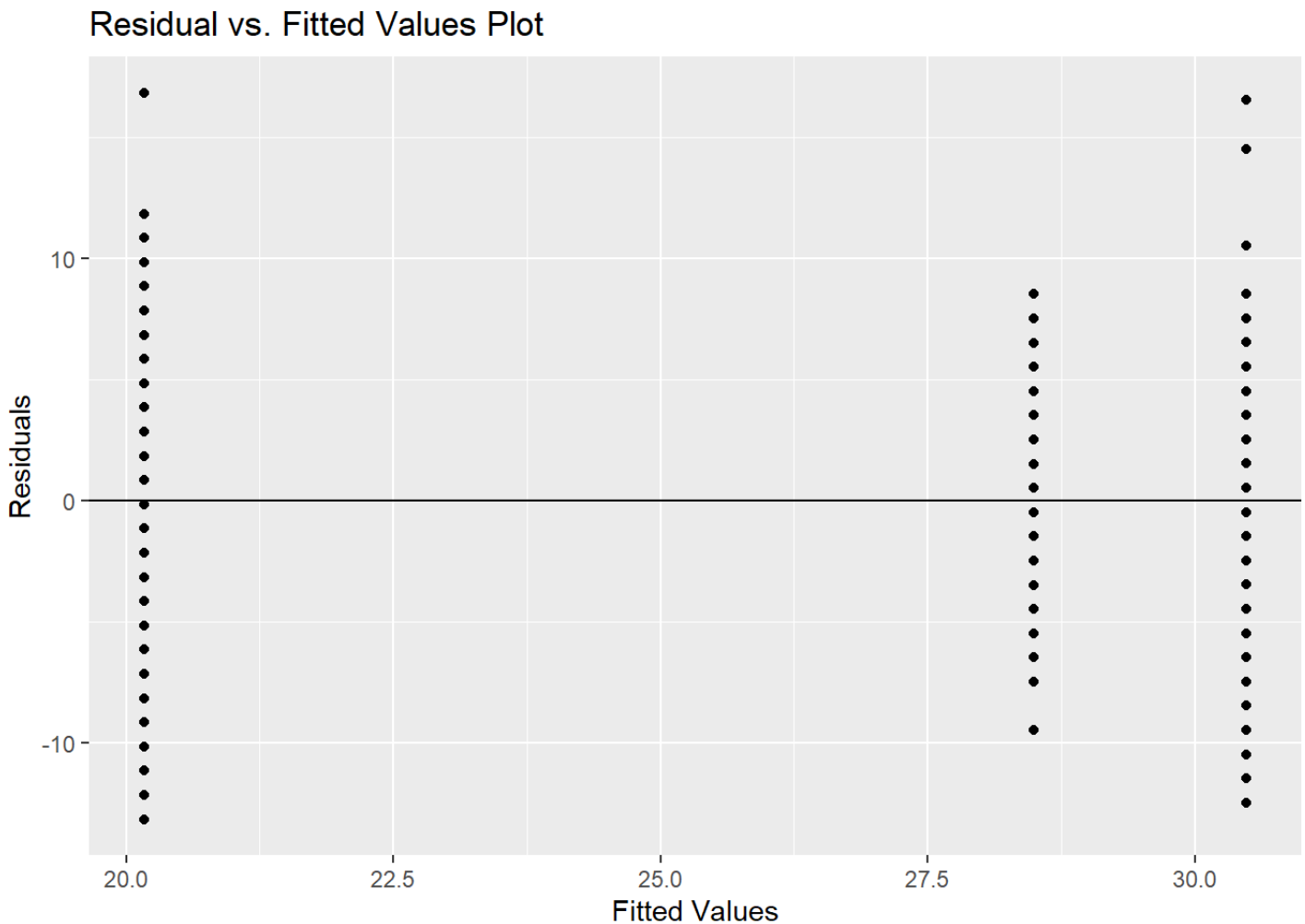
```
library(car)
library(stats)

leveneTest(MPG~factor(Country),data=cars2) # Most popular, but depends on Normal:

## Levene's Test for Homogeneity of Variance (center = median)
##           Df F value  Pr(>F)
## group    2  6.2318 0.00215 **
##           425
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

According to the Levene test, we reject the null hypothesis and would conclude that at least one variance is significantly different. We can also look at the residual plot.

```
ggplot(model, aes(.fitted, .resid)) +
  geom_point() +
  geom_hline(yintercept = 0)+
  labs(title='Residual vs. Fitted Values Plot', x='Fitted Values', y='Residuals')
```



IF we were able to FAIL TO REJECT this hypothesis and felt comfortable, we could do the ANOVA:

```
model<-aov(MPG~factor(Country), data=cars2)
summary(model)
```

```
##              Df Sum Sq Mean Sq F value Pr(>F)
## factor(Country)  2    8997     4498   172.2 <2e-16 ***
## Residuals      425   11105        26
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Our conclusion would be to reject the null hypothesis and assume that there is at least one mean that is significantly different. But remember, this test is NOT accurate since our variance are not the same.

A more appropriate test would be to perform the Welch's One-way ANOVA. Just as with testing two means, Welch also did an extension to the ANOVA with unequal variances (this does assume Normality or at least not a too big of a departure from Normality):

```
oneway.test(MPG~factor(Country), data = cars2, var.equal = FALSE)

##
## One-way analysis of means (not assuming equal variances)
##
## data:  MPG and factor(Country)
## F = 174.97, num df = 2.0, denom df = 177.3, p-value < 2.2e-16
```

This is still testing if there is a significant difference in the means, however, it is designed for unequal variances. We arrive at the same conclusion here...there is at least one significant difference in the mean MPG for cars made in US, Japan and Germany.

Although a one-way ANOVA is designed to assess whether or not there is a significant difference within the mean values of the response with respect to the different levels of the predictor variable, we can draw some parallel to the regression model. For example, if we have $k=4$, then we can let x_a , x_b , and x_c be 3 reference-coded dummy variables for the levels: a , b , c , and d . Note that we only have 3 dummy variables because one gets left out for the reference level, in this case it is d . The linear model is of the following form:

$$y = \beta_0 + \beta_a x_a + \beta_b x_b + \beta_c x_c + \varepsilon \quad (2.2)$$

If we define x_a as 1 if the observation belongs to level a and 0 otherwise, and the same definition for x_b and x_c , then this is called *reference-level coding* (this will change for effects-level coding). The predicted values in (2.2) is basically the predicted mean of the response within the 4 levels of the predictor variable.

- β_0 represents the mean of reference group, group d .
- $\beta_a, \beta_b, \beta_c$ all represent the *difference* in the respective group means compared to the reference level. Positive values thus reflect a group mean that is higher than the reference group, and negative values reflect a group mean lower than the reference group.
- ε is called the **error**.

A *one-way* ANOVA model only contains a single input variable of interest. Equation (2.2), while it has 3 dummy variable inputs, only contains a single nominal attribute. In 3, we will add more inputs to the equation via two-way ANOVA and multivariate regression models.

ANOVA is used to test the following hypothesis:

$$H_0 : \beta_a = \beta_b = \beta_c = 0 \quad (\text{i.e. all group means are equal})$$

$$H_0 : \beta_a \neq 0 \text{ or } \beta_b \neq 0 \text{ or } \beta_c \neq 0 \quad (\text{i.e. at least one is different})$$

Both the `lm()` function and the `aov()` function will provide the p-values to test the hypothesis above, the only difference between the two functions is that `lm()` will also provide the user with the coefficient of determination, R^2 , which tells you how much of the variation in y is accounted for by your categorical input.

```
cars_lm <- lm(MPG ~ factor(Country), data = cars2)
summary(cars_lm)
```

```
##
## Call:
## lm(formula = MPG ~ factor(Country), data = cars2)
##
## Residuals:
```

	Min	1Q	Median	3Q	Max
	-13.1647	-3.4900	-0.1647	2.8353	16.8353

```
##
## Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	28.4900	0.5112	55.735	<2e-16 ***
factor(Country)Japanese	1.9910	0.7694	2.588	0.01 **
factor(Country)US	-8.3253	0.6052	-13.757	<2e-16 ***

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.112 on 425 degrees of freedom
## Multiple R-squared:  0.4476, Adjusted R-squared:  0.445
## F-statistic: 172.2 on 2 and 425 DF,  p-value: < 2.2e-16
```

We can also confirm what we know about the predictions from ANOVA, that there are only k unique predictions from an ANOVA with k groups (the predictions being the group means), using the `predict` function.

```
cars2$predict <- predict(cars_lm, data = cars2)

cars2$resid_anova <- resid(cars_lm, data = cars2)

model_output <- cars2 %>% dplyr::select(Country, predict, resid_anova)

rbind(model_output[1:3,], model_output[255:257,], model_output[375:377,])

##      Country  predict resid_anova
## 1         US 20.16466  -13.164659
## 2         US 20.16466  -12.164659
## 3         US 20.16466  -12.164659
## 255 Japanese 30.48101   4.518987
## 256 Japanese 30.48101  -6.481013
## 257 Japanese 30.48101 -11.481013
## 375  Germany 28.49000  -9.490000
## 376  Germany 28.49000  -2.490000
## 377  Germany 28.49000   8.510000
```

A non-parametric version of the ANOVA test, the Kruskal-Wallis test, is also available. This test does NOT assume normality, but does need *similar* distributions among the different levels. Non-parametric tests do not have the same *statistical power* to detect differences between groups. **Statistical power** is the probability of detecting an effect, if there is a true effect present to detect. We should opt for these tests in situations where our data is ordinal or otherwise violates the assumptions of normality in ways that cannot be fixed by logarithmic or other similar transformation.

2.4.1 Python Code

Using statsmodels (linear regression approach to ANOVA):

```
import statsmodels.formula.api as smf

model = smf.ols("Sale_Price ~ C(Exter_Qual)", data = train).fit()
model.summary()
```

OLS Regression Results			
Dep. Variable:	Sale_Price	R-squared:	0.507
Model:	OLS	Adj. R-squared:	0.506
Method:	Least Squares	F-statistic:	701.8
Date:	Wed, 19 Jun 2024	Prob (F-statistic):	1.00e-313
Time:	11:43:36	Log-Likelihood:	-25346.
No. Observations:	2051	AIC:	5.070e+04
Df Residuals:	2047	BIC:	5.072e+04
Df Model:	3		
Covariance Type:	nonrobust		

	coef	std err	t	P> t 	[0.025	0.975]
Intercept	8.422e+04	1.07e+04	7.905	0.000	6.33e+04	1.05e+05
C(Exter_Qual) [T.Typical]	5.789e+04	1.08e+04	5.374	0.000	3.68e+04	7.9e+04
C(Exter_Qual) [T.Good]	1.447e+05	1.09e+04	13.310	0.000	1.23e+05	1.66e+05
C(Exter_Qual) [T.Excellent]	2.917e+05	1.24e+04	23.486	0.000	2.67e+05	3.16e+05
Omnibus:	698.567	Durbin-Watson:		2.057		
Prob(Omnibus):	0.000	Jarque-Bera (JB):		5159.626		
Skew:	1.404	Prob(JB):		0.00		
Kurtosis:	10.245	Cond. No.		21.4		

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
sm.api.stats.anova_lm(model, typ=2)
```

```
##                sum_sq      df          F          PR(>F)
## C(Exter_Qual)  6.691308e+12    3.0  701.831849  1.000132e-313
## Residual      6.505408e+12  2047.0          NaN          NaN
```

Another way of doing it in statsmodels:

```

sm.stats.oneway.anova_oneway(data = train['Sale_Price'], groups = train['Exter_Q
## <class 'statsmodels.stats.base.HolderTuple'>
## statistic = 701.8318492122419
## pvalue = 1.00013222764e-313
## df = (3.0, 2047.0)
## df_num = 3.0
## df_denom = 2047.0
## nobs_t = 2051.0
## n_groups = 4
## means = array([375904.17948718,  84219.39285714, 228909.64264706, 142107.3067
## nobs = array([ 78.,  28., 680., 1265.])
## vars_ = array([1.10988963e+10, 1.13439030e+09, 5.15642396e+09, 1.67638622e+09]
## use_var = 'equal'
## welch_correction = False
## tuple = (701.8318492122419, 1.00013222764e-313)

```

Using Scipy instead of statsmodels:

```

sp.stats.f_oneway(train['Sale_Price'][train['Exter_Qual'] == 'Excellent'],
                  train['Sale_Price'][train['Exter_Qual'] == 'Good'],
                  train['Sale_Price'][train['Exter_Qual'] == 'Typical'],
                  train['Sale_Price'][train['Exter_Qual'] == 'Fair'])

## F_onewayResult(statistic=701.8318492122418, pvalue=1.00013222764e-313)

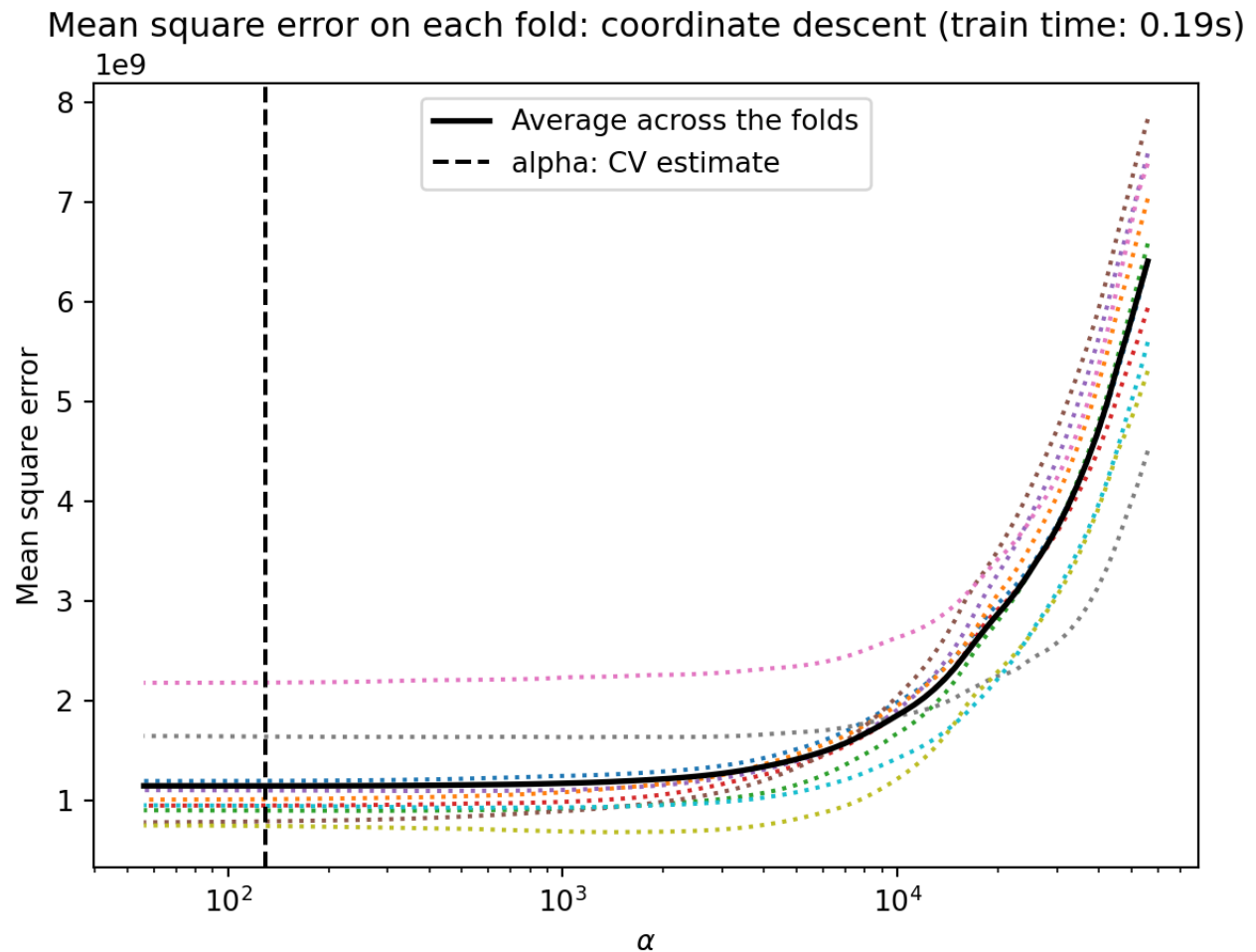
```

```
train['pred_anova'] = model.predict()
train['resid_anova'] = model.resid

train[['Sale_Price', 'pred_anova', 'resid_anova']].head(n = 10)
```

##	Sale_Price	pred_anova	resid_anova
## 0	232600	228909.642647	3690.357353
## 1	166000	228909.642647	-62909.642647
## 2	170000	142107.306719	27892.693281
## 3	252000	228909.642647	23090.357353
## 4	134000	142107.306719	-8107.306719
## 5	164700	228909.642647	-64209.642647
## 6	193500	142107.306719	51392.693281
## 7	118500	142107.306719	-23607.306719
## 8	94000	142107.306719	-48107.306719
## 9	111250	142107.306719	-30857.306719

```
sm.api.qqplot(train['resid_anova'])
plt.show()
```



```
sp.stats.shapiro(model.resid)
```

```
## ShapiroResult(statistic=0.9187809406211249, pvalue=7.319434407349789e-32)
```

```
sp.stats.levene(train['Sale_Price'][train['Exter_Qual'] == 'Excellent'],
                train['Sale_Price'][train['Exter_Qual'] == 'Good'],
                train['Sale_Price'][train['Exter_Qual'] == 'Typical'],
                train['Sale_Price'][train['Exter_Qual'] == 'Fair'])
```

```
## LeveneResult(statistic=76.87916723411712, pvalue=4.0352332459689874e-47)
```

```
sp.stats.fligner(train['Sale_Price'][train['Exter_Qual'] == 'Excellent'],  
                 train['Sale_Price'][train['Exter_Qual'] == 'Good'],  
                 train['Sale_Price'][train['Exter_Qual'] == 'Typical'],  
                 train['Sale_Price'][train['Exter_Qual'] == 'Fair'])
```

```
## FlignerResult(statistic=206.2591684986186, pvalue=1.8733883114714104e-44)
```

2.4.2 Kruskal-Wallis

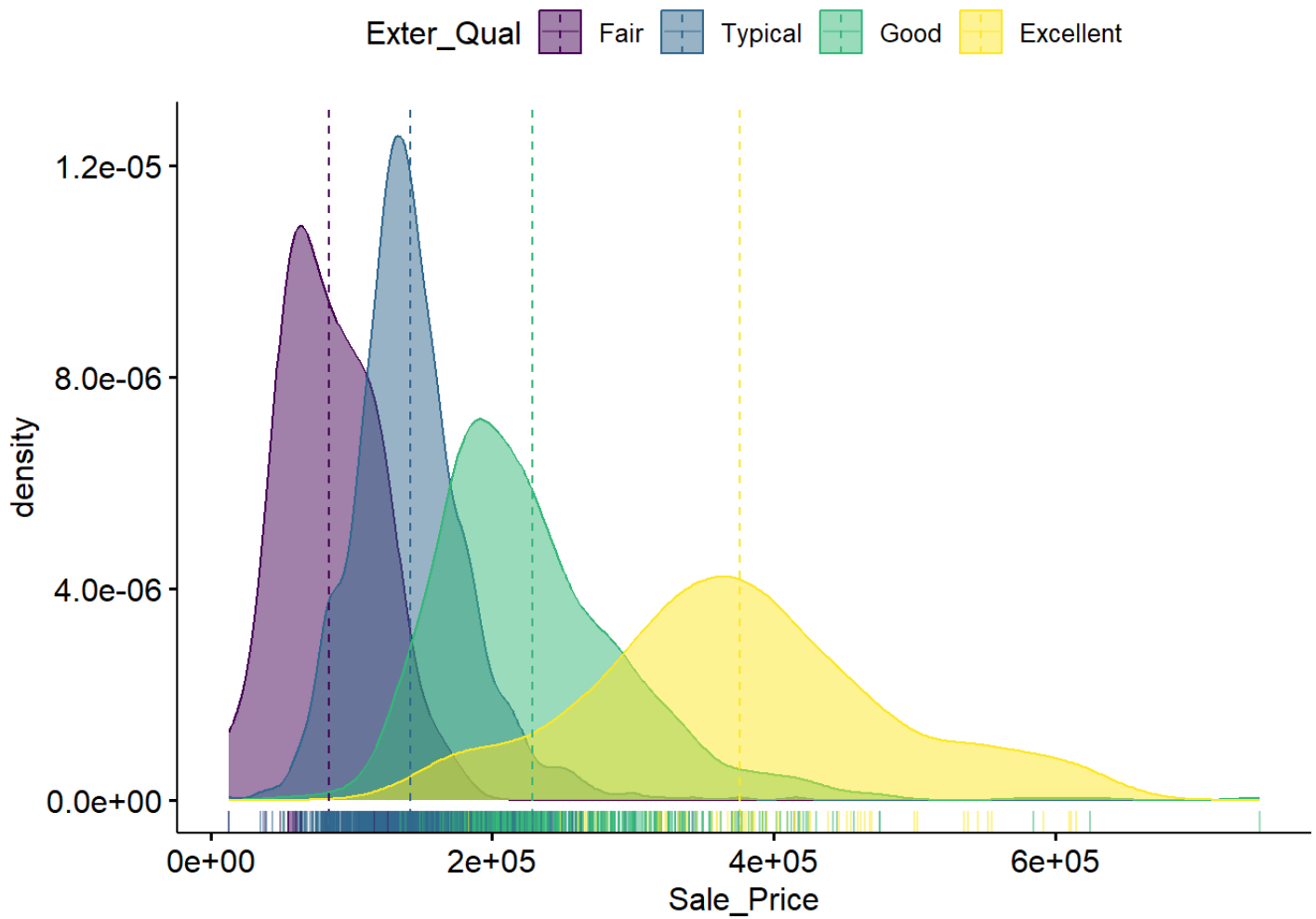
The Kruskal-Wallis test, proposed in 1952, is equivalent to a parametric one-way ANOVA where the data values have been replaced with their ranks (i.e. largest value = 1, second largest value = 2, etc.). When the data *is not* normally distributed but *is* identically distributed (having the same shape and variance), the Kruskal-Wallis test can be considered a test for differences in medians. If those identical distributions are also symmetric, then Kruskal-Wallis can be interpreted as testing for a difference in means. When the data is not identically distributed, or when the distributions are not symmetric, Kruskal-Wallis is a test of **dominance** between distributions. **Distributional dominance** is the notion that one group's distribution is located at larger values than another, probabilistically speaking. Formally, a random variable A has dominance over random variable B if $P(A \geq x) \geq P(B \geq x)$ for all x , and for some x , $P(A \geq x) > P(B \geq x)$.

We summarize this information in the following table:

Conditions	Interpretation of Significant Kruskal-Wallis Test
Group distributions are identical in shape, variance, and symmetric	Difference in means
Group distributions are identical in shape, but not symmetric	Difference in medians
Group distributions are not identical in shape, variance, and are not symmetric	Difference in location. (distributional dominance)

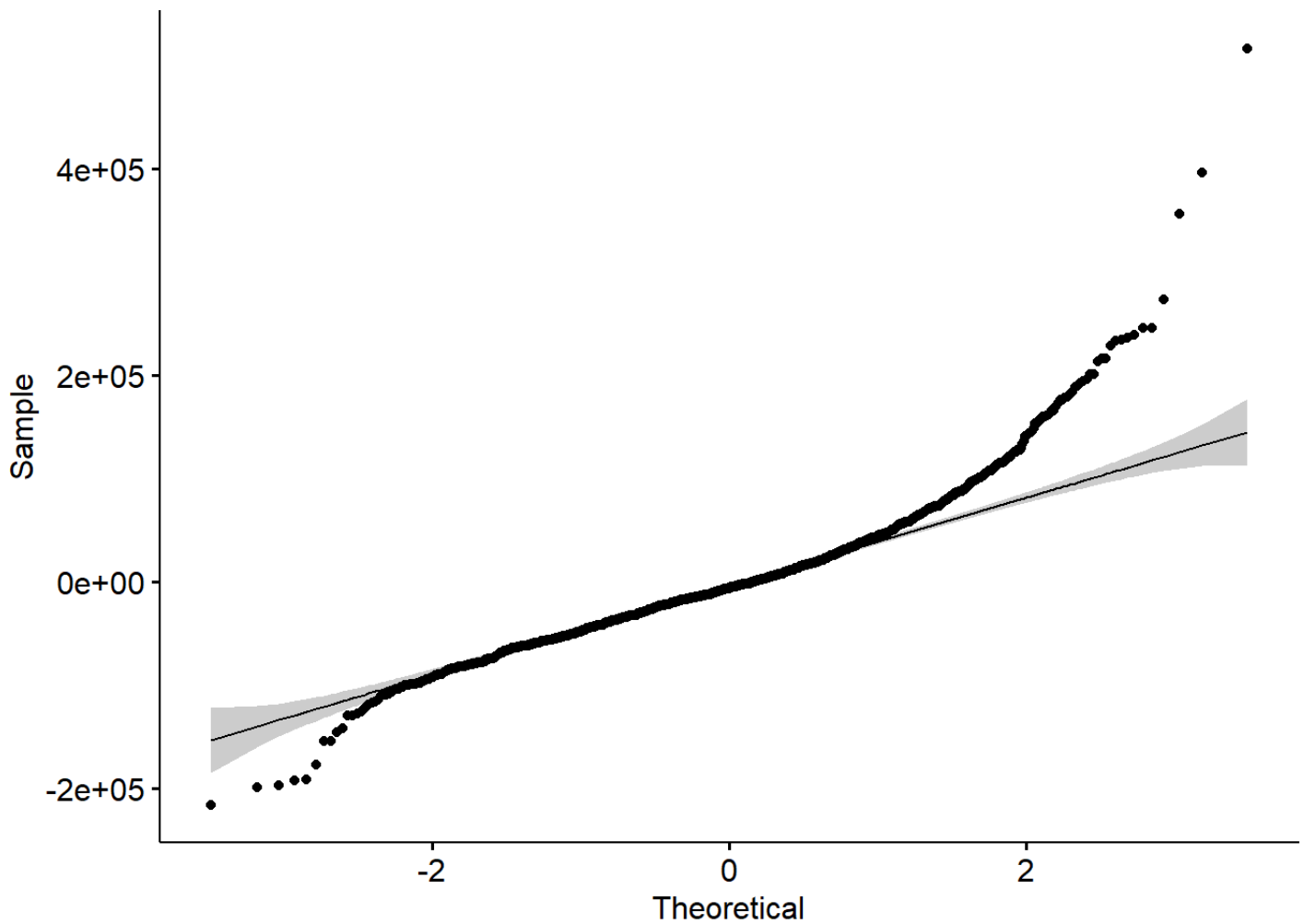
Implementing the Kruskal-Wallis test in R is simple:

```
ggdensity(train, x = "Sale_Price",  
          add = "mean", rug = TRUE,  
          color = "Exter_Qual", fill = "Exter_Qual")
```



```
model<-aov(Sale_Price ~ factor(Exter_Qual), data = train)
```

```
ggqqplot(residuals(model))
```



```
fligner.test(Sale_Price ~ Exter_Qual, data = train) # DOES NOT depend on Normality
```

```
##
```

```
## Fligner-Killeen test of homogeneity of variances
```

```
##
```

```
## data: Sale_Price by Exter_Qual
```

```
## Fligner-Killeen:med chi-squared = 206.26, df = 3, p-value < 2.2e-16
```

```
kruskal.test(MPG~factor(Country),data=cars2)
```

```
##  
## Kruskal-Wallis rank sum test  
##  
## data: MPG by factor(Country)  
## Kruskal-Wallis chi-squared = 197.74, df = 2, p-value < 2.2e-16
```

Our conclusion would be that the distribution of sale price is different across different levels of exterior quality.

2.4.3 Python Code

```
sp.stats.kruskal(train['Sale_Price'][train['Exter_Qual'] == 'Excellent'],  
                 train['Sale_Price'][train['Exter_Qual'] == 'Good'],  
                 train['Sale_Price'][train['Exter_Qual'] == 'Typical'],  
                 train['Sale_Price'][train['Exter_Qual'] == 'Fair'])  
  
## KruskalResult(statistic=975.9781472622338, pvalue=2.9251120247488444e-211)
```

2.5 ANOVA Post-hoc Testing

After performing an ANOVA and learning that there is a difference between the groups of data, our next natural question ought to be *which groups of data are different, and how?* In order to explore this question, we must first consider the notion of **experimentwise error**. When conducting multiple hypothesis tests simultaneously, the experimentwise error rate is the proportion of time you expect to make an error in at least one test.

Let's suppose we are comparing grocery spending on 4 different credit card rewards programs. If we'd like to compare the rewards programs pairwise, that entails 6 different hypothesis tests (each is a two-sample t-test). If we keep a confidence level of $\alpha = 0.05$ and subsequently view "being wrong in one test" as a random event happening with probability $p = 0.05$ then our probability of being wrong *in at least* one test out of 6 could be as great as 0.26!

To control this experiment-wise error rate, we must lower our significance thresholds to account for it. Alternatively, we can view this as an adjustment of our p-values higher while keeping our significance threshold fixed as usual. This is typically the approach taken, as we prefer to fix our significance thresholds in accordance with previous literature or industry standards. There are many methods of adjustment that have been proposed over the years for this purpose. Here, we consider a few popular methods: Tukey's test (when a basic ANOVA is done), Games-Howell test (when Welch's ANOVA is done), Dunn's test (when Kruskal-Wallis test is done) for pairwise comparisons and Dunnett's test (or Mann-Whitney test with Bonferroni correction) for control group comparisons.

2.5.1 Tukey-Kramer

If our objective is to compare each group to every other group then Tukey's test of honest significant differences, also known as the Tukey-Kramer test is probably the most widely-available and popular corrections in practice (should ONLY be used if you do the basic ANOVA). However, it should be noted that Tukey's test *should not* be used if one does not plan to make all pairwise comparisons. If only a subset of comparisons are of interest to the user (like comparisons only to a control group) then one should opt for the Dunnett or a modified Bonferroni correction.

To employ Tukey's HSD in R, we must use the `aov()` function to create our ANOVA object rather than the `lm()` function. The output of the test shows the difference in means and the p-value for testing the null hypothesis that the means are equal (i.e. that the differences are equal to 0).

```
cars_aov <- aov(MPG~factor(Country), data = cars2)
tukey.cars <- TukeyHSD(cars_aov)
print(tukey.cars)

## Tukey multiple comparisons of means
## 95% family-wise confidence level
##
## Fit: aov(formula = MPG ~ factor(Country), data = cars2)
##
## $`factor(Country)`
##
```

	diff	lwr	upr	p adj
Japanese-Germany	1.991013	0.1813164	3.800709	0.0269593
US-Germany	-8.325341	-9.7486718	-6.902011	0.0000000
US-Japanese	-10.316354	-11.8687997	-8.763908	0.0000000

```
par(mar=c(4,10,4,2))
plot(tukey.cars, las = 1)
```

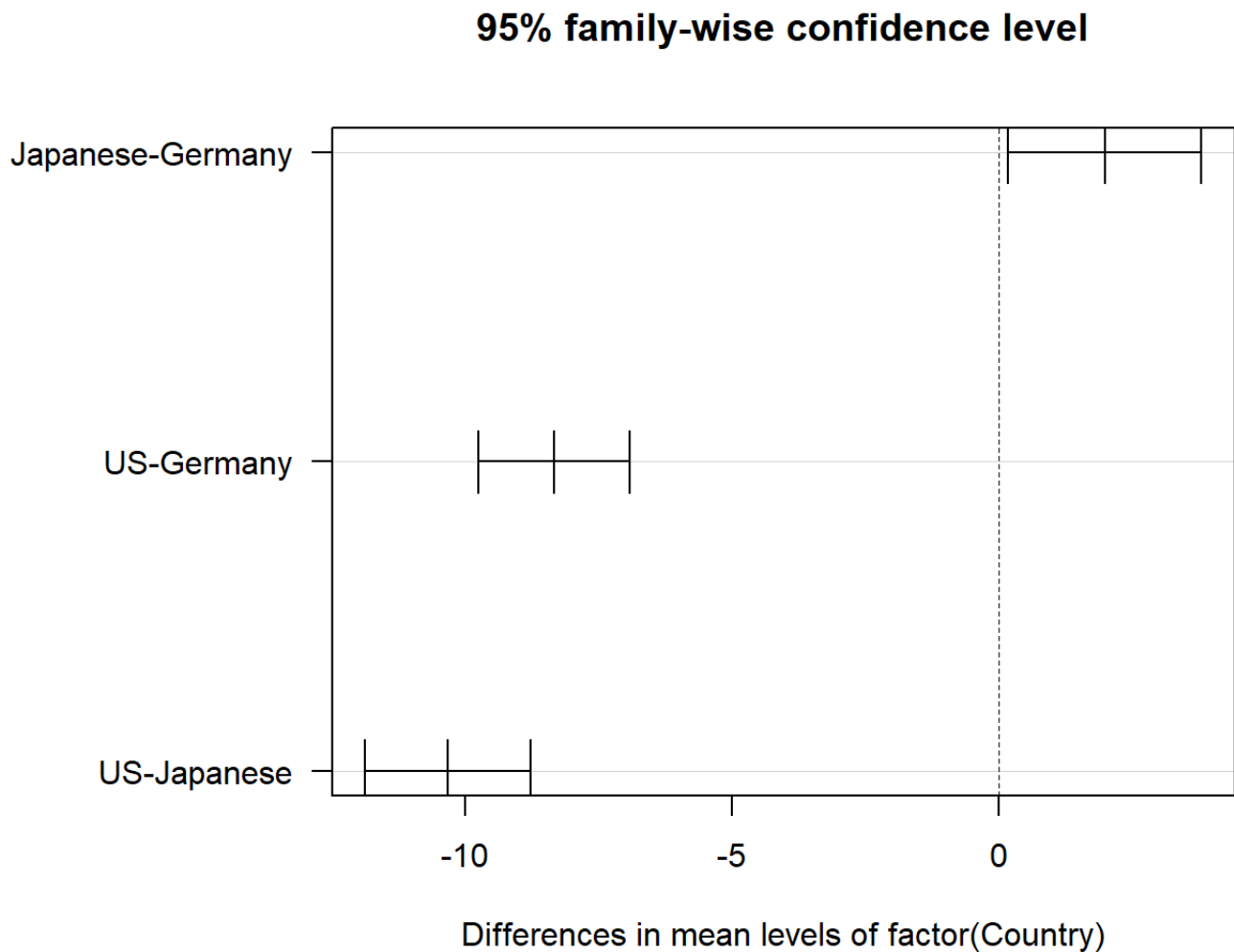


Figure 2.6: Confidence intervals for mean differences adjusted via Tukey-Kramer

The p-values in this table have been adjusted higher to account for the possible experimentwise error rate. For every pairwise comparison shown, we reject the null hypothesis and conclude that the mean MPG is different among the US, Japanese and German made cars. Furthermore, Figure 2.6 shows us experiment-wise (family-wise) adjusted confidence intervals for the differences in means for each pair. The plot option `las=1` guides the axis labels. Type `?par` for a list of plot options for base R, including an explanation of `las`.

If we utilized the Welch's ANOVA (assumed variances were NOT equal), then we would use the Games-Howell test:

```
library(PMCMRplus)
```

```
## Warning: package 'PMCMRplus' was built under R version 4.3.3
```

```
cars_aov <- aov(MPG~factor(Country), data=cars2)
gh.cars <- gamesHowellTest(cars_aov)
summary(gh.cars)
```

```
##
```

```
## Pairwise comparisons using Games–Howell test
```

```
## data: MPG by factor(Country)
```

```
## alternative hypothesis: two.sided
```

```
## P value adjustment method: none
```

```
## H0
```

```
##               q value   Pr(>|q|)
## Japanese - Germany == 0    3.547   0.035479   *
## US - Germany == 0         -22.859 5.3291e-15 ***
## US - Japanese == 0        -19.164 4.3188e-14 ***
```

```
## ---
```

```
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The conclusion from the Games-Howell test is similar to Tukey's test.

However, if we use Kruskal-Wallis test, then we would use the Dunn's test for multiple comparisons:

```
library(dunn.test)
```

```
## Warning: package 'dunn.test' was built under R version 4.3.3
```

```
dunn.cars <- dunn.test(cars2$MPG, cars2$Country, kw=T, method="bonferroni")
```

```
## Kruskal-Wallis rank sum test
##
## data: x and group
## Kruskal-Wallis chi-squared = 197.7403, df = 2, p-value = 0
##
##
##
## Comparison of x by group
## (Bonferroni)
## Col Mean-|
## Row Mean | Germany Japanese
## -----+-----
## Japanese | -1.147428
##          | 0.3768
##          |
## US       | 10.95670 11.38301
##          | 0.0000* 0.0000*
##
## alpha = 0.05
## Reject Ho if p <= alpha/2
```

This is a nonparametric test and does not have as much power as the previous multiple comparisons. Be sure you use the correct test based on what you see in the data.

2.5.2 Dunnett's Test

If the plan is to make fewer comparisons, specifically just $k - 1$ comparisons where k is the number of groups in your data (indicating you plan to compare all the groups to one specific group, usually the control group), then Dunnett's test would be preferable to the Tukey-Kramer test. If all pairwise comparisons are not made, the Tukey-Kramer test is overly

conservative, creating a confidence level that is much lower than specified by the user. Dunnett's test factors in fewer comparisons and thus should not be used for tests of all pairwise comparisons.

To use Dunnett's test (should only be used with basic ANOVA), we must add the `DescTools` package to our library. The control group to which all other groups will be compared is designated by the `control=` option.

```
cars2$group<-ifelse(cars2$Country=="US","aUS",cars2$Country)
dunnettTest(x = cars2$MPG, g = factor(cars2$group))

##              aUS
## Germany < 2e-16
## Japanese 4.4e-16
```

In the output from Dunnett's test, we see that Japanese and German made cars are significantly different than US made cars (note that this does not compare Japanese to German!).

If we are not able to assume equal variances, we can use the Welch two-sample test to compare each group to the control and then adjust the p-value using Bonferroni.

We can use this same idea to to the Wilcoxon test for a nonparametric version (then adjust using Bonferroni).

```
summary(welchManyOneTTest(x = cars2$MPG, g = factor(cars2$group),p.adjust.method=

##
## Pairwise comparisons using Welch's t-test
```

```
## data: cars2$MPG and factor(cars2$group)
```

```
## alternative hypothesis: two.sided
```

```
## P value adjustment method: bonferroni
```

```
## H0
```

```
##               t value    Pr(>|t|)
## Germany - aUS == 0   16.164 < 2.22e-16 ***
## Japanese - aUS == 0   13.551 < 2.22e-16 ***
```

```
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(manyOneUtest(x = cars2$MPG, g = factor(cars2$group), p.adjust.method="bonferroni"))
```

```
##
```

```
## Pairwise comparisons using Wilcoxon, Mann, Whittney U-test
```

```
## for multiple comparisons with one control
```

```
## data: cars2$MPG and factor(cars2$group)
```

```
## alternative hypothesis: two.sided

## P value adjustment method: bonferroni

## H0

##
##          z value    Pr(>|z|)
## Germany - aUS == 0 -11.565 < 2.22e-16 ***
## Japanese - aUS == 0 -10.677 < 2.22e-16 ***

## ---

## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

2.5.3 Python Code

Tukey-Kramer test:

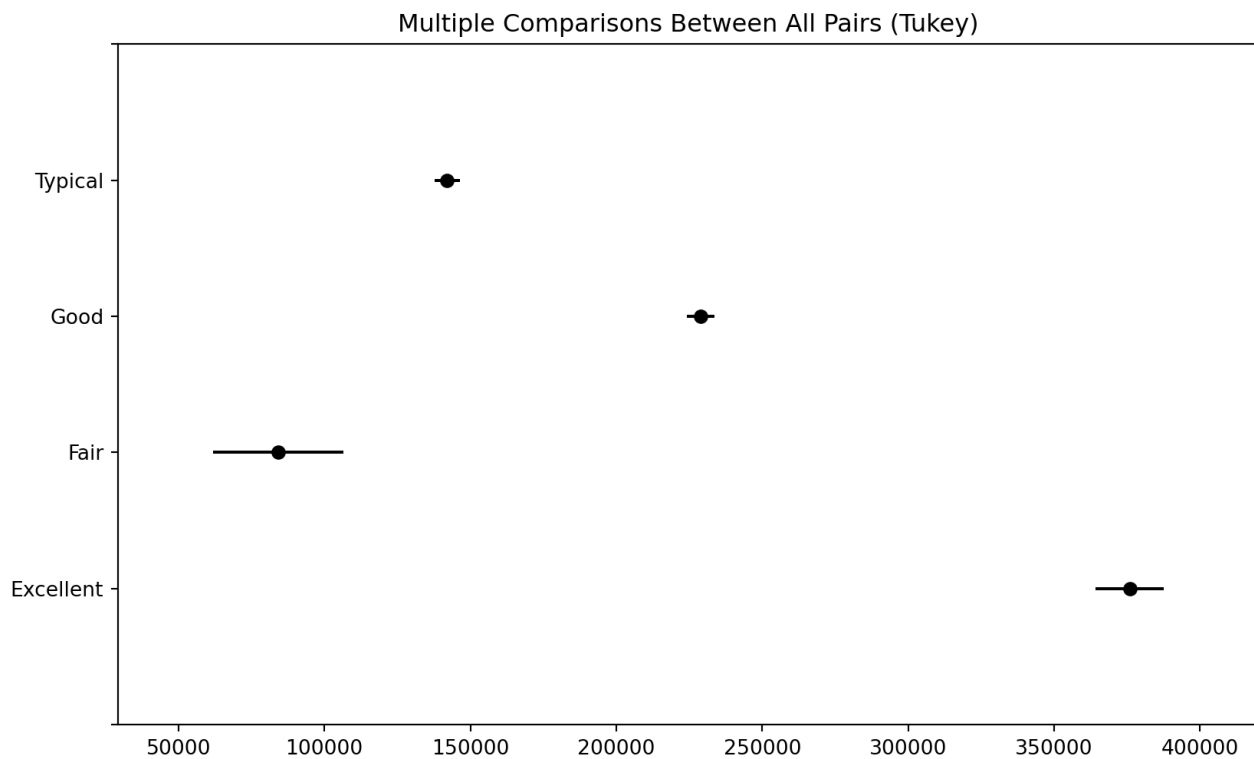
```
import statsmodels.stats.multicomp as mc

comp = mc.MultiComparison(train['Sale_Price'], train['Exter_Qual'])
ph_res = comp.tukeyhsd(alpha = 0.05)
ph_res.summary()
```

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
Excellent	Fair	-291684.7866	0.0	-323617.1589	-259752.4144	True
Excellent	Good	-146994.5368	0.0	-164322.0969	-129666.9768	True
Excellent	Typical	-233796.8728	0.0	-250707.122	-216886.6235	True
Fair	Good	144690.2498	0.0	116739.8693	172640.6303	True
Fair	Typical	57887.9139	0.0	30194.3052	85581.5226	True
Good	Typical	-86802.3359	0.0	-93694.6435	-79910.0283	True

```
ph_res.plot_simultaneous()
```



No Dunnett's test in Python. The team at statsmodels is supposedly “working on it”.

2.6 Pearson Correlation

ANOVA is used to formally test the relationship between a categorical variable and a continuous variable. To formally test the (linear) relationship between two continuous attributes, we introduce **Pearson correlation**, commonly referred to as simply **correlation**. Correlation is a number between -1 and 1 which measures the strength of a linear relationship between two continuous attributes.

Negative values of correlation indicate a *negative linear relationship*, meaning that as one of the variables increases, the other tends to decrease. Similarly, positive values of correlation indicate a *positive linear relationship* meaning that as one of the variables increases, the other tends to increase. Absolute values of correlation equal to 1 indicate a perfect linear relationship. For example, if our data had a column for “mile time in minutes” and a column for “mile time in seconds”, these two columns would have a correlation of 1 due to the fact that there are 60 seconds in a minute. A correlation value near 0 indicates that the variables have no linear relationship.

It's important to emphasize that Pearson correlation is only designed to detect *linear* associations between variables. Even when a correlation between two variables is 0, the two variables may still have a very clear association, whether it be quadratic, cyclical, or some other nonlinear pattern of association. Figure 2.7 illustrates all of these statements. On top of each scatter plot, the correlation coefficient is shown. The middle row of this figure aims to illustrate that a perfect correlation has nothing to do with the *magnitude* or *slope* of the relationship. In the center image, middle row, we note that the correlation is undefined for any pair that includes a constant variable. In that image, the value of y is constant across the sample. Equation (2.3) makes this mathematically clear.

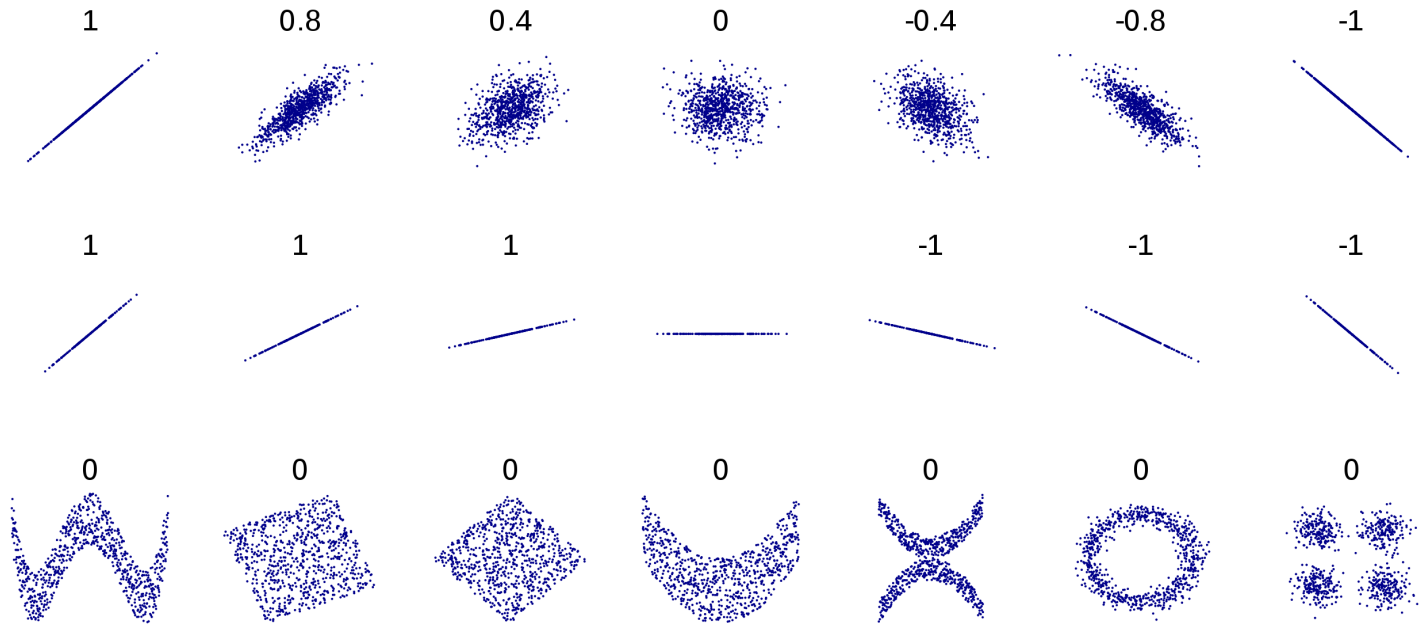


Figure 2.7: Examples of relationships and their associated correlations

The population correlation parameter is denoted ρ and estimated by the sample correlation, denoted as r . The formula for the sample correlation between columns of data $\mathbf{x}$ and $\mathbf{y}$ is

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}. \quad (2.3)$$

Note that with *centered* variable vectors $\mathbf{x}_c$ and $\mathbf{y}_c$ this formula becomes much cleaner with linear algebra notation:

$$r = \frac{\mathbf{x}_c^T \mathbf{y}_c}{\|\mathbf{x}_c\| \|\mathbf{y}_c\|}. \quad (2.4)$$

It is interesting to note that Equation (2.4) is identical to the formula for the cosine of the angle between two vectors. While this geometrical relationship does not benefit our intuition¹, it is noteworthy nonetheless.

Pearson's correlation can be calculated in R with the built in `cor()` function, with the two continuous variables as input:

```
cor(train$Gr_Liv_Area,train$Sale_Price)
```

```
## [1] 0.698509
```

2.6.1 Statistical Test

To test the statistical significance of correlation, we use a t-test with the null hypothesis that the correlation is equal to 0:

$$H_0 : \rho = 0$$

$$H_a : \rho \neq 0$$

If we can reject the null hypothesis, then we declare a significant linear association between the two variables. The `cor.test()` function in R will perform the test:

```
cor.test(train$Gr_Liv_Area,train$Sale_Price)
```

```
##
```

```
## Pearson's product-moment correlation
```

```
##
```

```
## data: x and y
```

```
## t = 44.185, df = 2049, p-value < 2.2e-16
```

```
## alternative hypothesis: true correlation is not equal to 0
```

```
## 95 percent confidence interval:
```

```
## 0.6756538 0.7200229
```

```
## sample estimates:
```

```
## cor
```

```
## 0.698509
```

We conclude that `Gr_Liv_Area` has a linear association with `Sale_Price`.

It must be noted that this t-test for Pearson's correlation is not free from assumptions. In fact, there are 4 assumptions that must be met, and they are detailed in Section [2.7.1](#).

2.6.2 Effect of Anomalous Observations

One final nuance that is important to note is the effect of anomalous observations on correlation. In Figure [2.8](#) we display 30 random 2-dimensional data points (x, y) with no linear relationship.

```
set.seed(11)
x <- rnorm(30)
y <- rnorm(30)
plot(x, y)
```

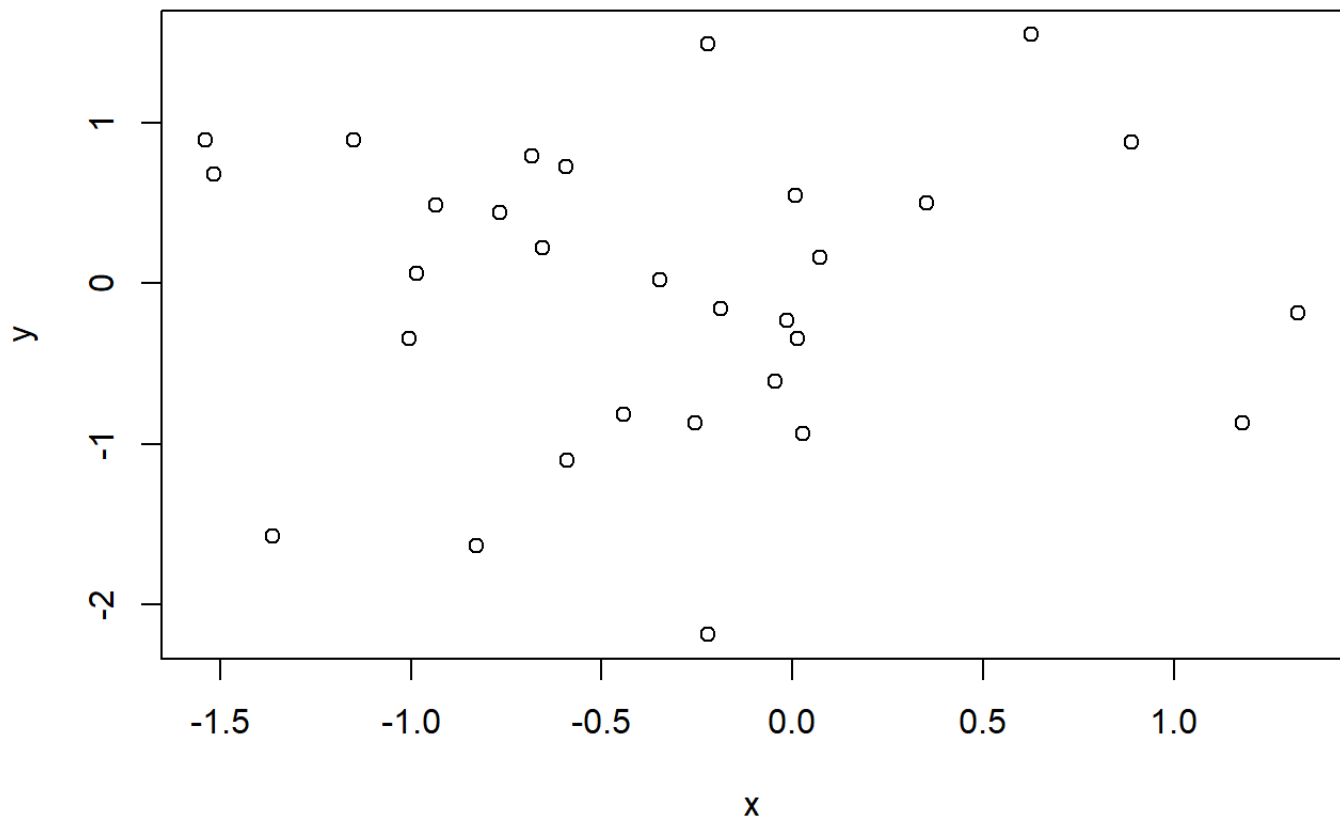


Figure 2.8: The variables x and y have no correlation

The correlation is not exactly zero (we wouldn't expect perfection from random data) but it is very close at 0.002.

```
cor.test(x,y)
```

```
##  
## Pearson's product-moment correlation  
##  
## data:  x and y  
## t = 0.012045, df = 28, p-value = 0.9905  
## alternative hypothesis: true correlation is not equal to 0  
## 95 percent confidence interval:  
## -0.3582868  0.3622484  
## sample estimates:  
##          cor  
## 0.002276214
```

Next, we'll add a single anomalous observation to our data and see how it affects both the correlation value and the correlation test.

```
x[31] = 4  
y[31] = 50  
cor.test(x,y)
```

```
##  
## Pearson's product-moment correlation  
##  
## data:  x and y  
## t = 5.803, df = 29, p-value = 2.738e-06  
## alternative hypothesis: true correlation is not equal to 0  
## 95 percent confidence interval:  
##  0.5115236 0.8631548  
## sample estimates:  
##          cor  
## 0.7330043
```

The correlation jumps to 0.73 from 0.002 and is declared strongly significant! Figure [2.9](#) illustrates the new data. This simple example shows why exploratory data analysis is so important! If we don't explore our data and detect anomalous observations, we might improperly declare relationships are significant when they are driven by a single observation or a small handful of observations.

```
plot(x,y)
```

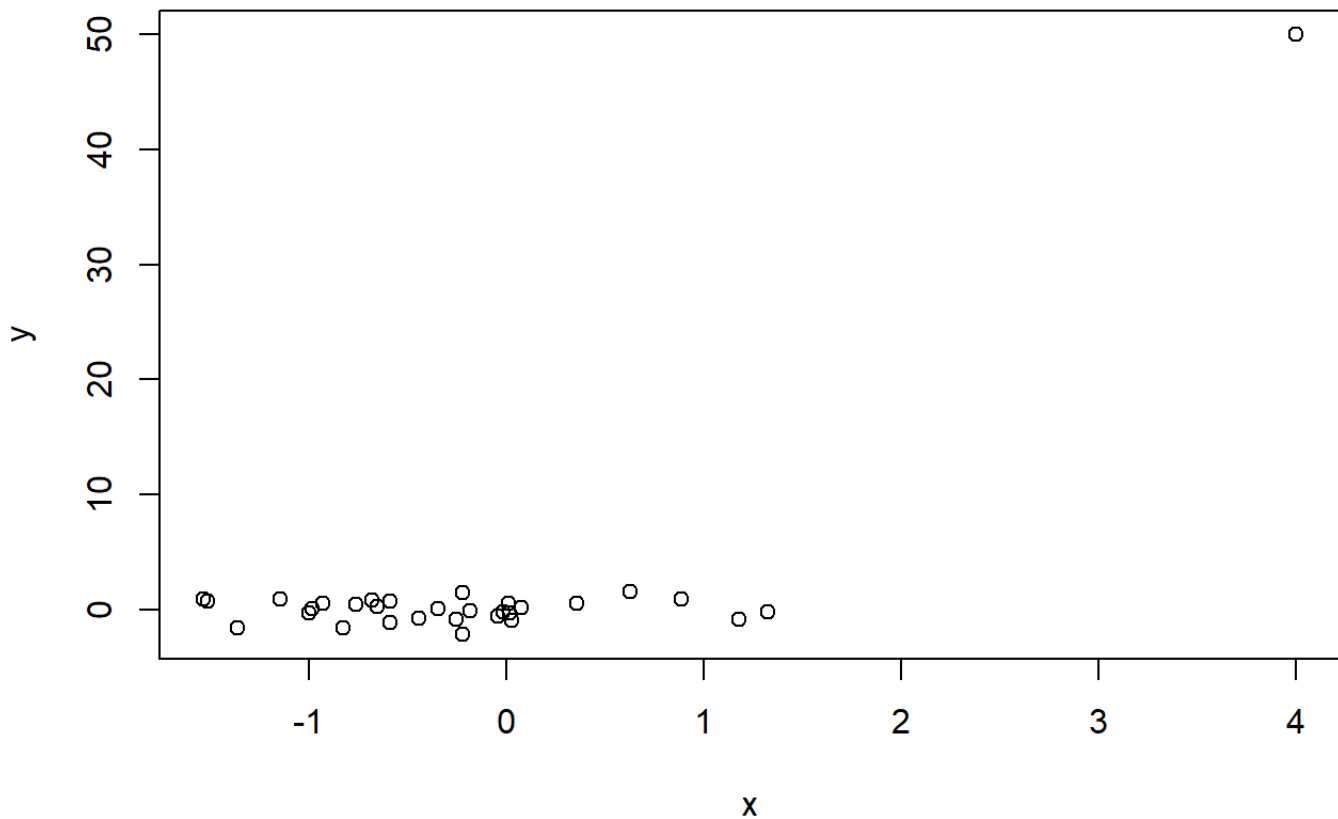


Figure 2.9: A single anomalous observation creates strong correlation (0.73) where there previously was none

2.6.3 The Correlation Matrix

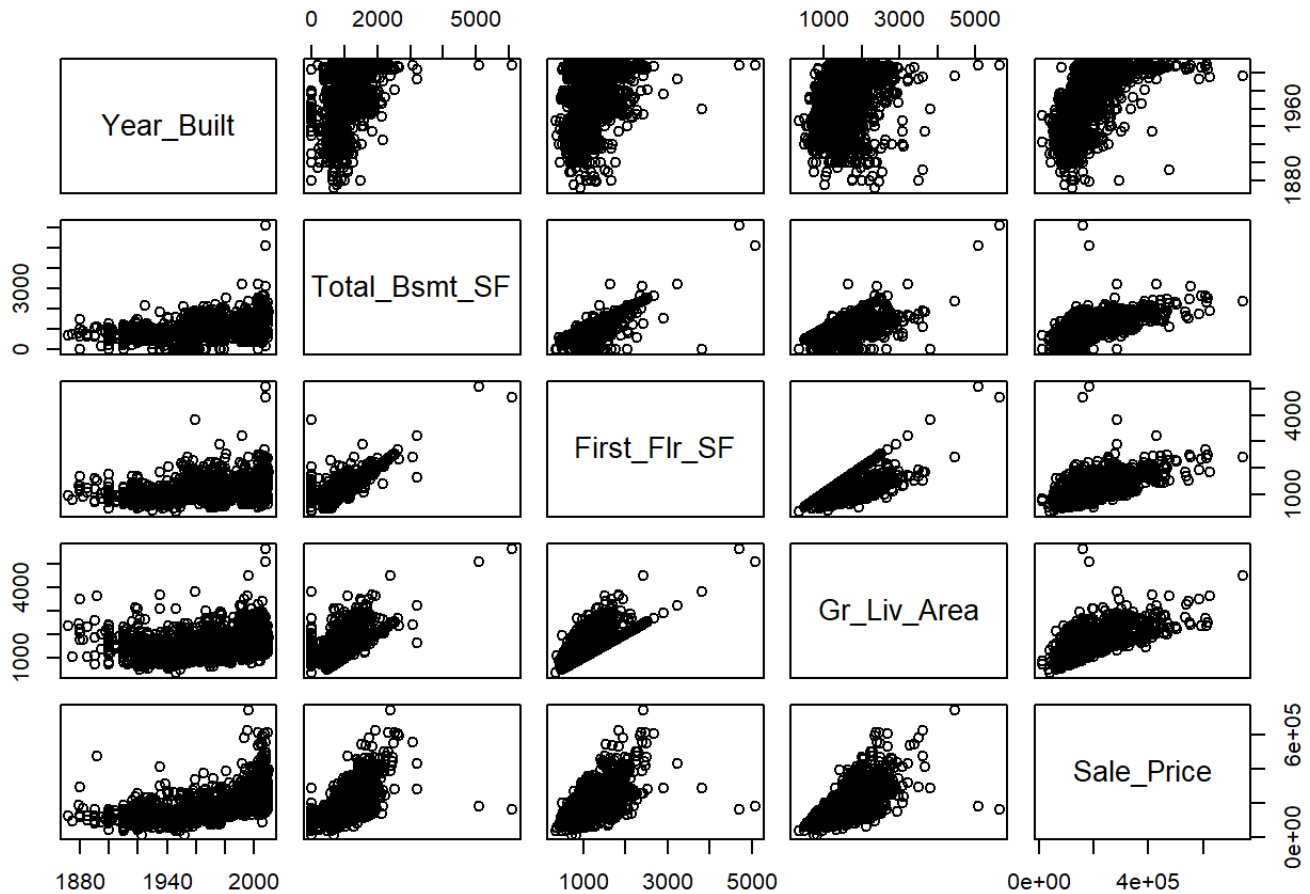
It's common to consider and calculate all pairwise correlations between variables in a dataset. If many attributes share a high degree of mutual correlation, this can cause problems for regression as will be discussed in Chapter 5. The pairwise correlations are generally arranged in an array called the **correlation matrix**, where the $(i, j)^{th}$ entry is the correlation between the i^{th} variable and j^{th} variable in your list. To compute the correlation matrix, we again use the `cor()` function.

```
cor(train[, c('Year_Built', 'Total_Bsmt_SF', 'First_Flr_SF', 'Gr_Liv_Area', 'Sale_Pr
```

```
##           Year_Built Total_Bsmt_SF First_Flr_SF Gr_Liv_Area Sale_Price
## Year_Built      1.0000000      0.4037104      0.3095407      0.2454325      0.5668889
## Total_Bsmt_SF  0.4037104      1.0000000      0.8120419      0.4643838      0.6276502
## First_Flr_SF   0.3095407      0.8120419      1.0000000      0.5707205      0.6085229
## Gr_Liv_Area    0.2454325      0.4643838      0.5707205      1.0000000      0.6985090
## Sale_Price     0.5668889      0.6276502      0.6085229      0.6985090      1.0000000
```

Not surprisingly, we see strong positive correlation between the square footage of the basement and that of the first floor, and also between all of the area variables and the sale price. As demonstrated by Figures 2.7 and 2.9, raw correlation values can be misleading and it's unwise to calculate them without a scatter plot for context. The `pairs()` function in base R provides a simple matrix of scatterplots for this purpose.

```
pairs(train[, c('Year_Built', 'Total_Bsmt_SF', 'First_Flr_SF', 'Gr_Liv_Area', 'Sale_I
```



2.6.4 Python Code

Pearson's correlation

```
np.corrcoef(train['Gr_Liv_Area'], train['Sale_Price'])
```

```
## array([[1.          , 0.69850904],
##        [0.69850904, 1.          ]])
```

Statistical test for Correlation

```
sp.stats.pearsonr(train['Gr_Liv_Area'], train['Sale_Price'])
```

```
## PearsonRResult(statistic=0.6985090408804115, pvalue=4.195282462397299e-300)
```

Correlation Matrix

```
np.corrcoef(train[['Year_Built', 'Total_Bsmt_SF', 'First_Flr_SF', 'Gr_Liv_Area',
```

```
## array([[1.          , 0.40371038, 0.3095407 , 0.24543253, 0.56688895],
##        [0.40371038, 1.          , 0.81204187, 0.46438378, 0.62765021],
##        [0.3095407 , 0.81204187, 1.          , 0.57072054, 0.60852293],
##        [0.24543253, 0.46438378, 0.57072054, 1.          , 0.69850904],
##        [0.56688895, 0.62765021, 0.60852293, 0.69850904, 1.          ]])
```

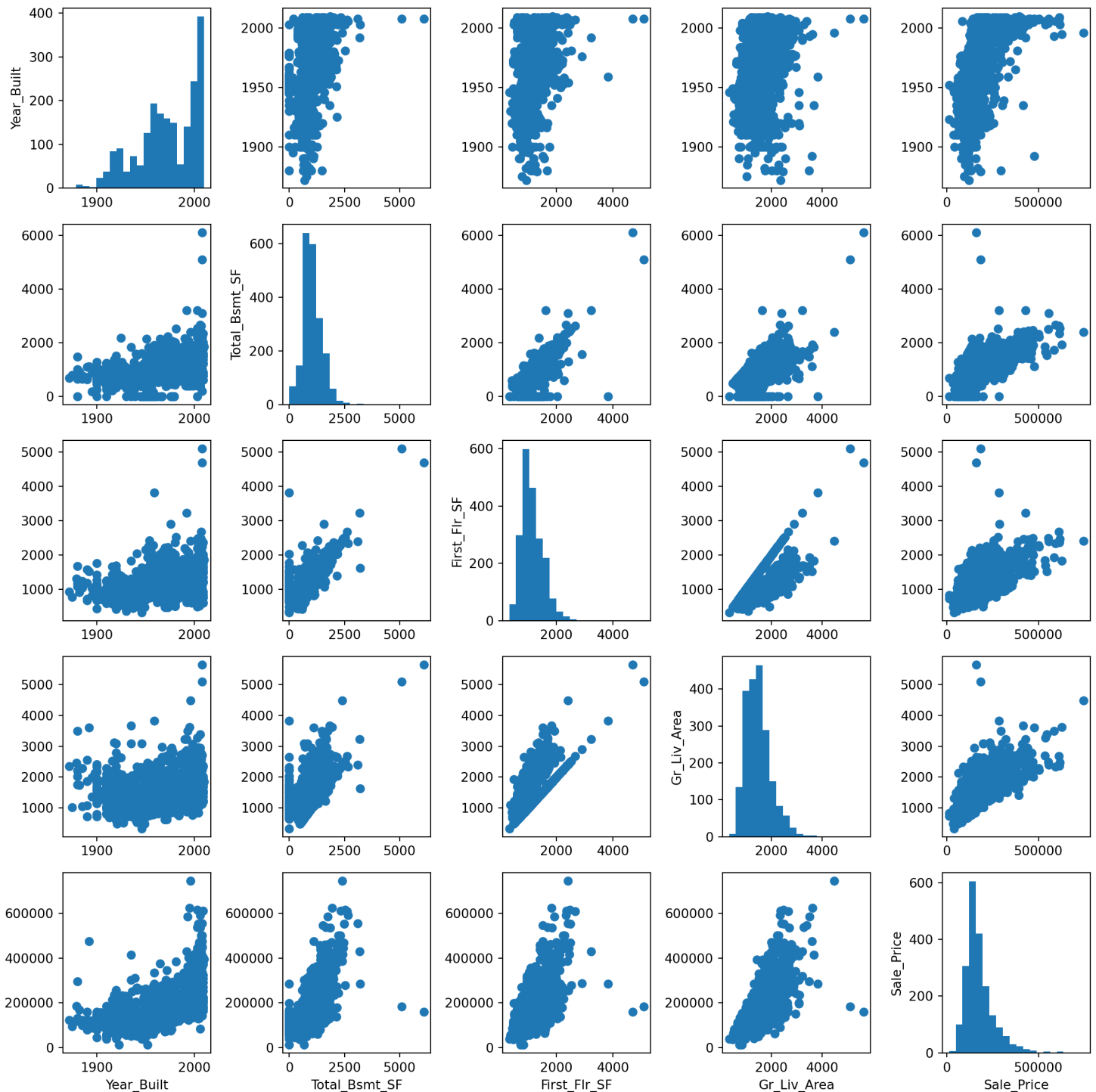
```
import itertools

# Selecting relevant columns
columns = ['Year_Built', 'Total_Bsmt_SF', 'First_Flr_SF', 'Gr_Liv_Area', 'Sale_Pri

# Create a pair plot
fig, axes = plt.subplots(nrows=len(columns), ncols=len(columns), figsize=(12, 12))

for i, col1 in enumerate(columns):
    for j, col2 in enumerate(columns):
        ax = axes[i, j]
        if i == j:
            ax.hist(train[col1], bins=20)
            ax.set_ylabel(col1)
        else:
            ax.scatter(train[col2], train[col1])
        if i == len(columns) - 1:
            ax.set_xlabel(col2)

plt.tight_layout()
plt.show()
```



2.7 Simple Linear Regression

After learning that two variables share a linear relationship, the next question is natural: *what is that relationship?* How much, on average, should we expect one variable to change as the other changes by a single unit? Simple linear regression answers this question by creating a linear equation that best represents the relationship in the sense that it minimizes the squared error between the observed data and the model predictions (i.e. the sum of the squared residuals). The simple linear regression equation is typically written

$$y = \beta_0 + \beta_1 x + \varepsilon \quad (2.5)$$

where β_0 , the **intercept**, gives the expected value of y when $x = 0$ and β_1 , the **slope** gives the expected change in y for a one-unit increase in x . The **error**, ε is the amount each individual y differs from the population line (we would not expect all values of y to fall directly on the line). When we use a sample of data to estimate the true population line, we get our prediction equation or $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x$. Residuals from the predicted line is defined as $e = y - \hat{y}$. *Ordinary Least Squares seeks to minimize the sum of squared residuals or sum of squared error*. That objective is known as a **loss function**. The sum of squared error (SSE) or equivalently the mean squared error (MSE) loss functions are by far the most popular loss functions for continuous prediction problems.

We should note that SSE is not the *only* loss function at our disposal. Minimizing the *mean absolute error* (MAE) is common in situations with a highly skewed response variable (squaring very large errors gives those observations in the tail too much influence on the regression as we will later discuss). Using MAE to drive our loss function gives us predictions that are conditional *medians* of the response, given the input data. Other loss functions, like Huber's M function, are also used to handle problems with influential observations as discussed in Chapter 5.

As we mentioned in Section 2.1, a simple linear regression serves two purposes:

1. to predict the expected value of y for each value of x and
2. to explain *how* y is expected to change for a unit change in x .

In order to accurately use a regression for the second purpose, however, we must first meet assumptions with our data.

2.7.1 Assumptions of Linear Regression

Linear regression, in particular the hypothesis tests that are generally performed as part of linear regression, has 4 assumptions:

1. The expected value of y is linear in x (proper model specification).
2. The random errors are independent.
3. The random errors are normally distributed.
4. The random errors have equal variance (homoskedasticity).

It must now be noted that these assumptions are also in effect for the test of Pearson's correlation in Section 2.6.1, because the tests in simple linear regression are mathematically equivalent to that test. When these assumptions are not met, another approach to testing the significance of a linear relationship should be considered. The most common non-parametric approach to testing for an association between two continuous variables is Spearman's correlation. Spearman's correlation does not limit its findings to linear relationships; any monotonic relationship (one that is always increasing or always decreasing) will cause Spearman's correlation to be significant. Similar to the approach taken by Kruskal-Wallis, Spearman's correlation replaces the data with its ranks and computes Pearson's correlation on the ranks. The same `cor` and `cor.test()` functions can be used; simply specify the `method='spearman'` option.

```
cor.test(train$Gr_Liv_Area,train$Sale_Price, method = 'spearman')
```

```
## Warning in cor.test.default(x, y, ...): Cannot compute exact p-value with tie:
```

```
##
## Spearman's rank correlation rho
##
## data:  x and y
## S = 408364087, p-value < 2.2e-16
## alternative hypothesis: true rho is not equal to 0
## sample estimates:
##      rho
## 0.7160107
```

2.7.2 Testing for Association

The statistical test of correlation is mathematically equivalent to testing the hypothesis that the slope parameter in Equation (2.5) is zero. This t-test is part of the output from any linear regression function, like `lm()` which we saw in Section 2.4. Let's confirm this using the example from the Section 2.6.1 where we investigate the relationship between `Gr_Liv_Area` and `Sale_Price`. Again, the t-test in the output tests the following hypothesis:

$$H_0 : \beta_1 = 0$$

$$H_a : \beta_1 \neq 0$$

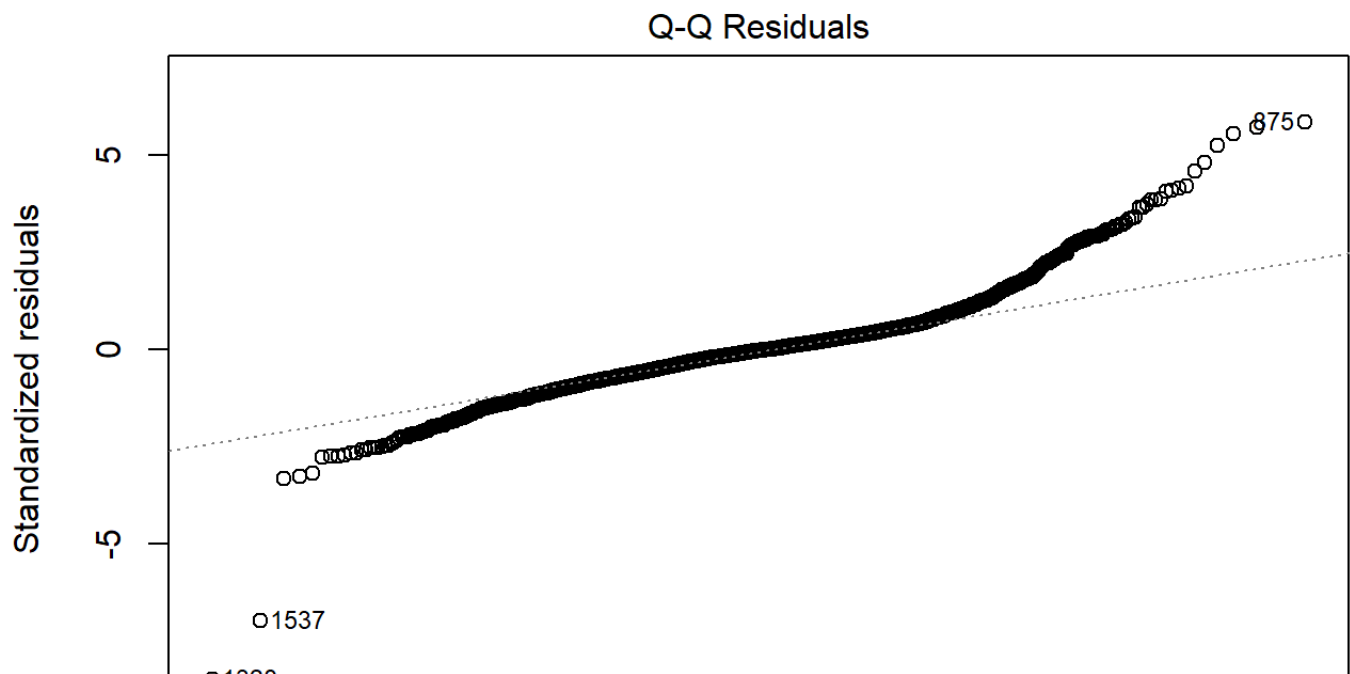
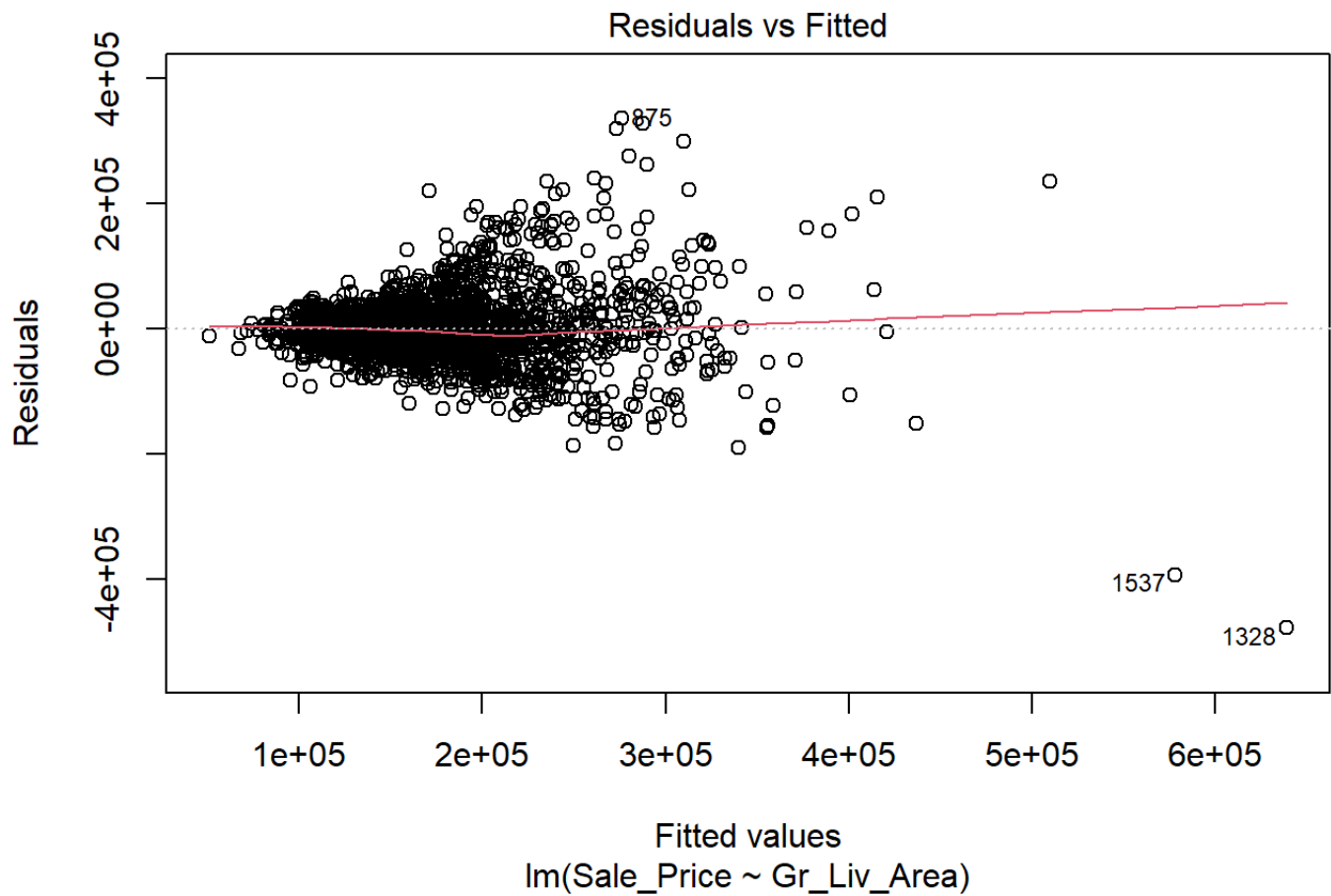
The first thing we will do after creating the linear model is check our assumption using the default plots from `lm()`. From these four plots we will be most interested in the first two.

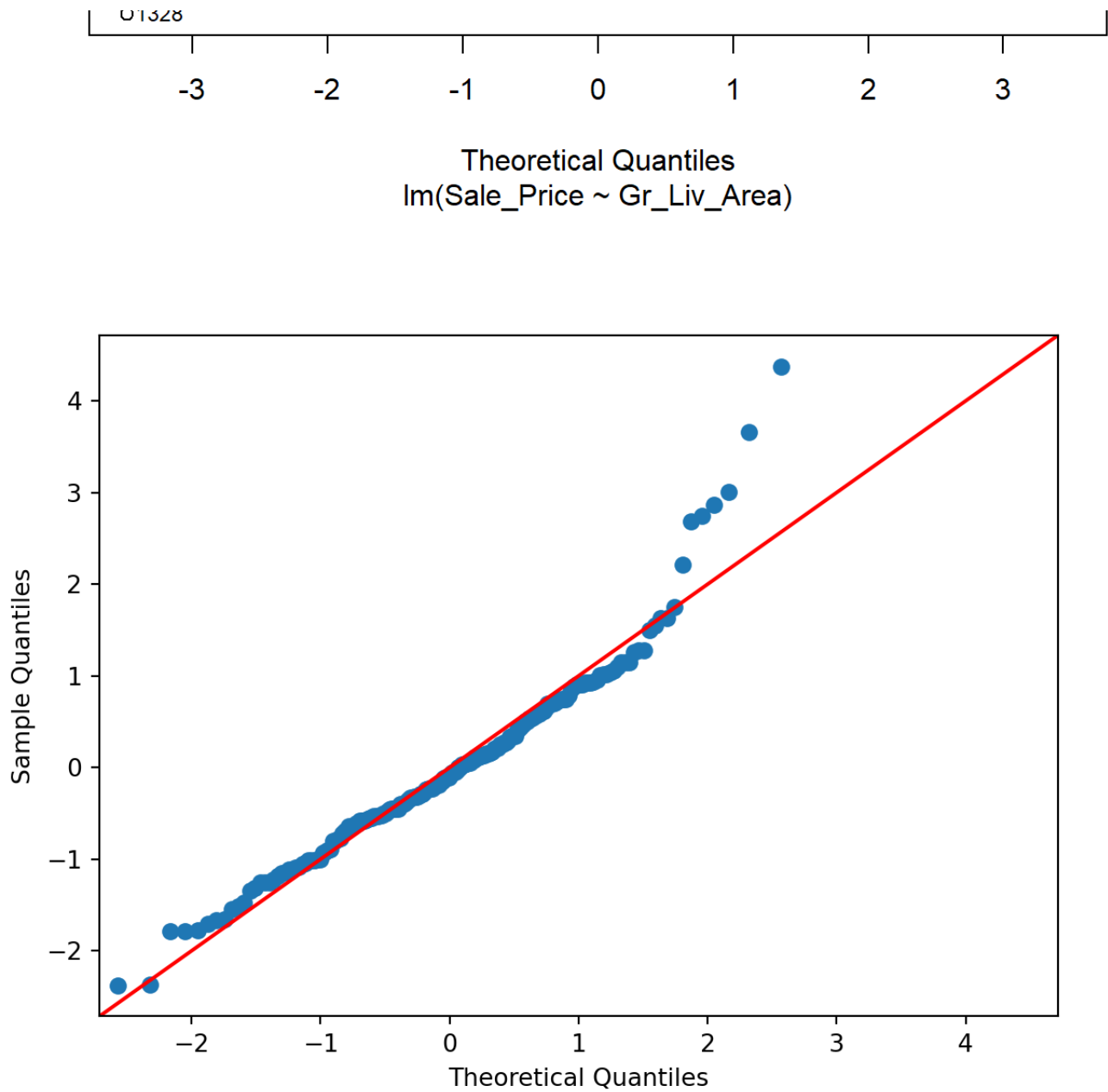
In the top left plot, we are visually checking for homoskedasticity. We'd like to see the variability of the points remain constant from left to right on this chart, indicating that the errors have constant variance for each value of y . We do not want to see any fan shapes in this chart. Unfortunately, we do see just that: the variability of the errors is much smaller for smaller values of Sale Price than it is for larger values of Sale Price.

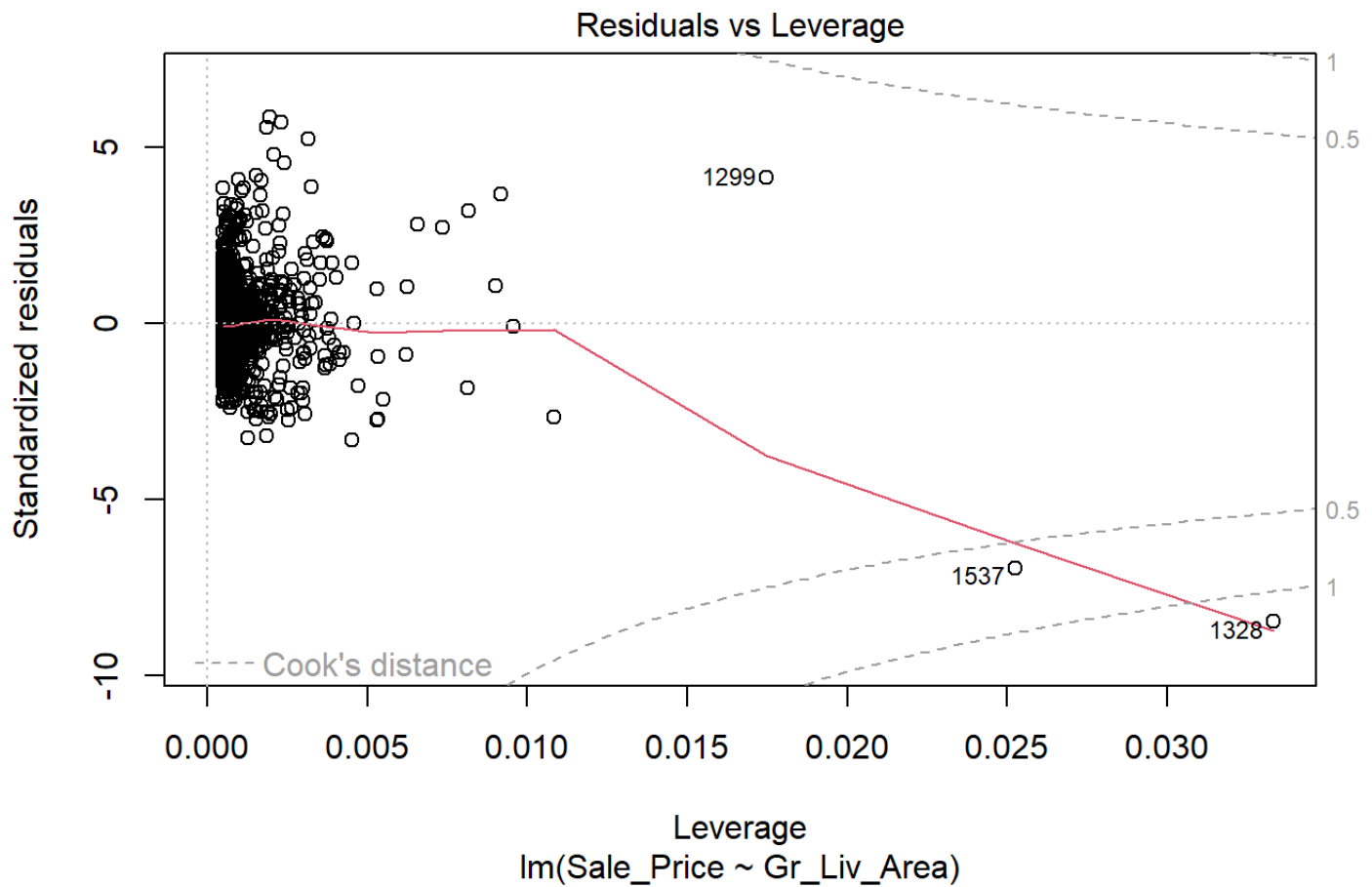
In the top right plot, we are visually checking for normality of errors. We'd like to see the QQ-plot indicate normality with all the points roughly following the line. Unfortunately, we do not see that here. The errors do not appear to be normally distributed.

```
slr <- lm(Sale_Price ~ Gr_Liv_Area, data=train)
```

```
plot(slr)
```







Despite the violation of assumptions, let's continue examining the output from this regression in order to practice our interpretation of it.

```
summary(slr)
```

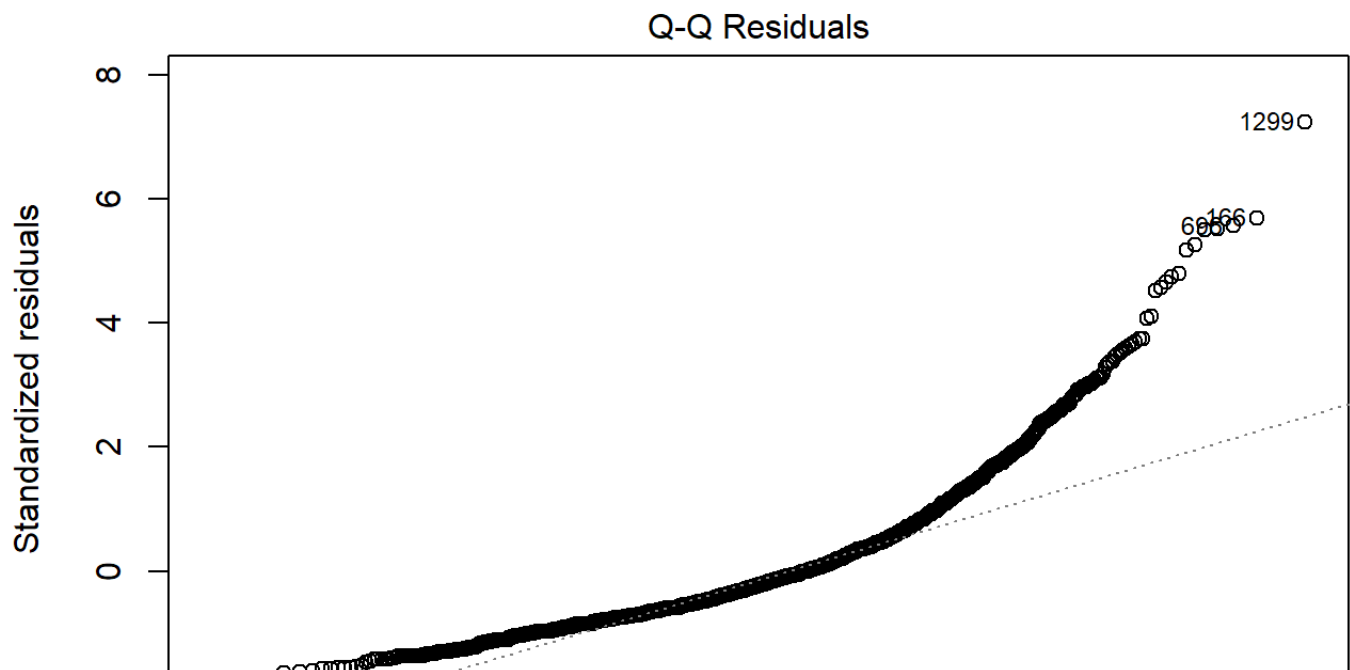
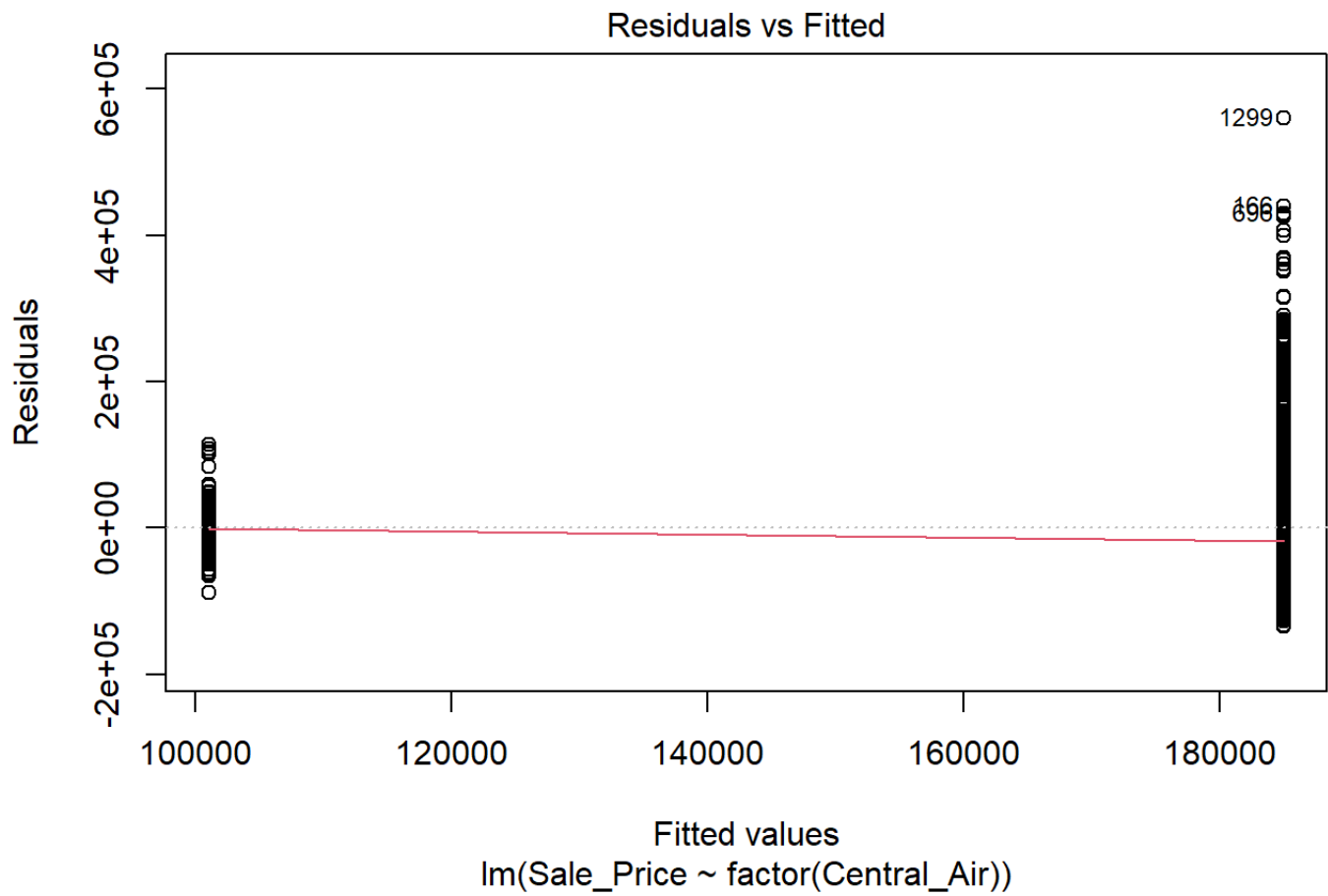
```
##
## Call:
## lm(formula = Sale_Price ~ Gr_Liv_Area, data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -478762  -30030   -1405   22273  335855
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 14045.872   3942.503   3.563 0.000375 ***
## Gr_Liv_Area   110.726     2.506  44.185 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 57430 on 2049 degrees of freedom
## Multiple R-squared:  0.4879, Adjusted R-squared:  0.4877
## F-statistic: 1952 on 1 and 2049 DF, p-value: < 2.2e-16
```

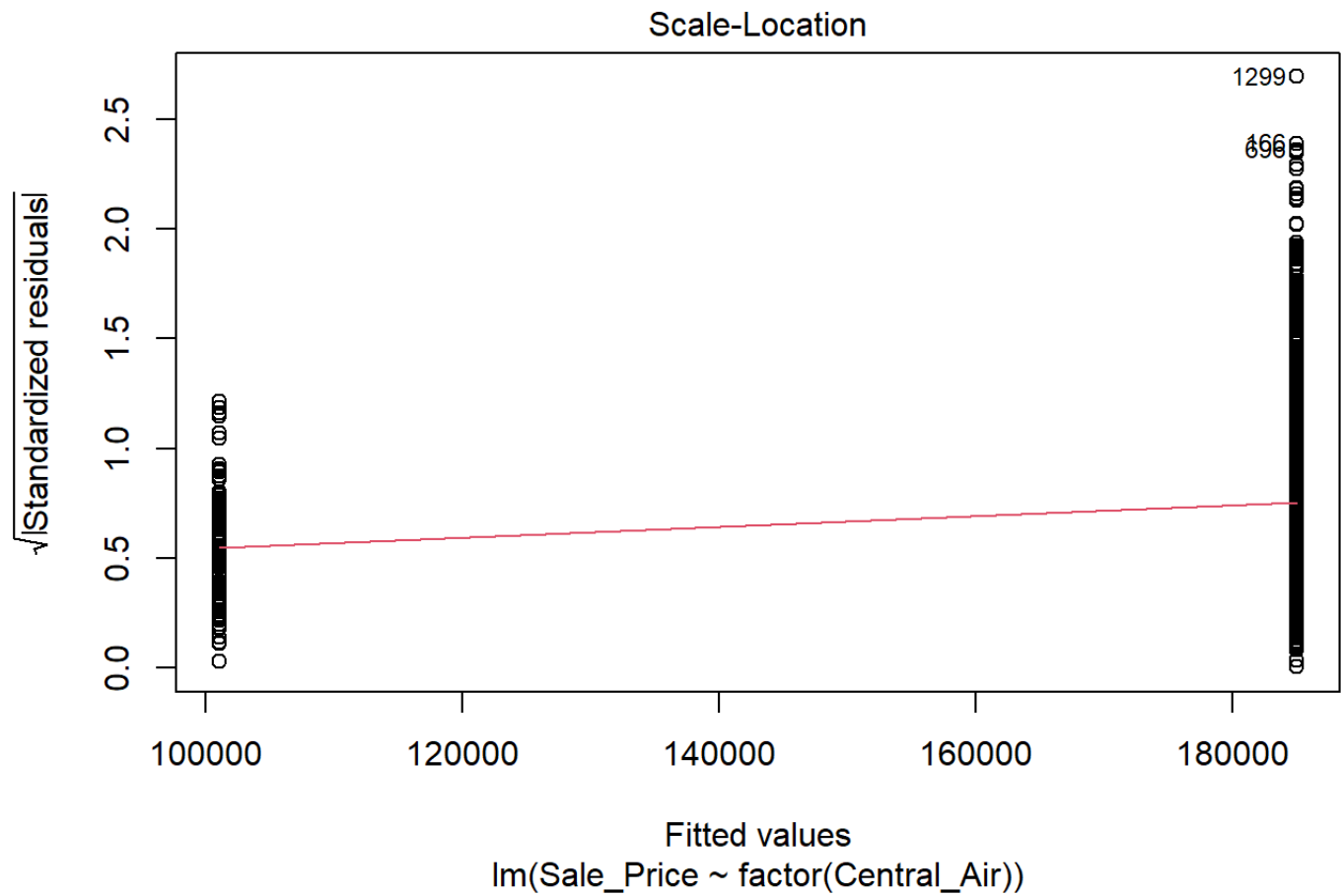
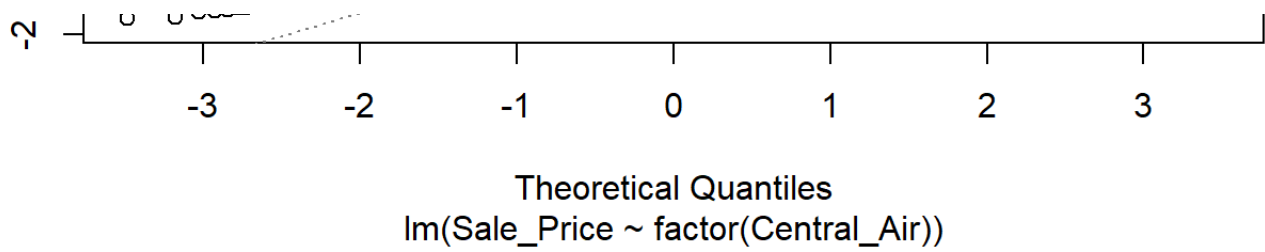
The first thing we're likely to examine in the coefficient table is the p-value for `Gr_Liv_Area`. It is strongly significant (in fact, it's the same t-value and p-value as we saw for the `cor.test` as these tests are mathematically equivalent), indicating that there is an association between the size of the home and the sale price. Furthermore, the *parameter estimate* is 110.73 indicating that we'd expect the price of a home to increase by \$110.73 for every additional square foot of living space. Because of the linearity of the model, we can extend this slope estimate to any unit change in x . For example, it might be difficult to think in terms of single square feet when comparing houses, so we might prefer to use a 100 square-foot change and report our conclusion as follows: For each additional 100 square feet of living area, we expect the house price to increase by \$11,0726.

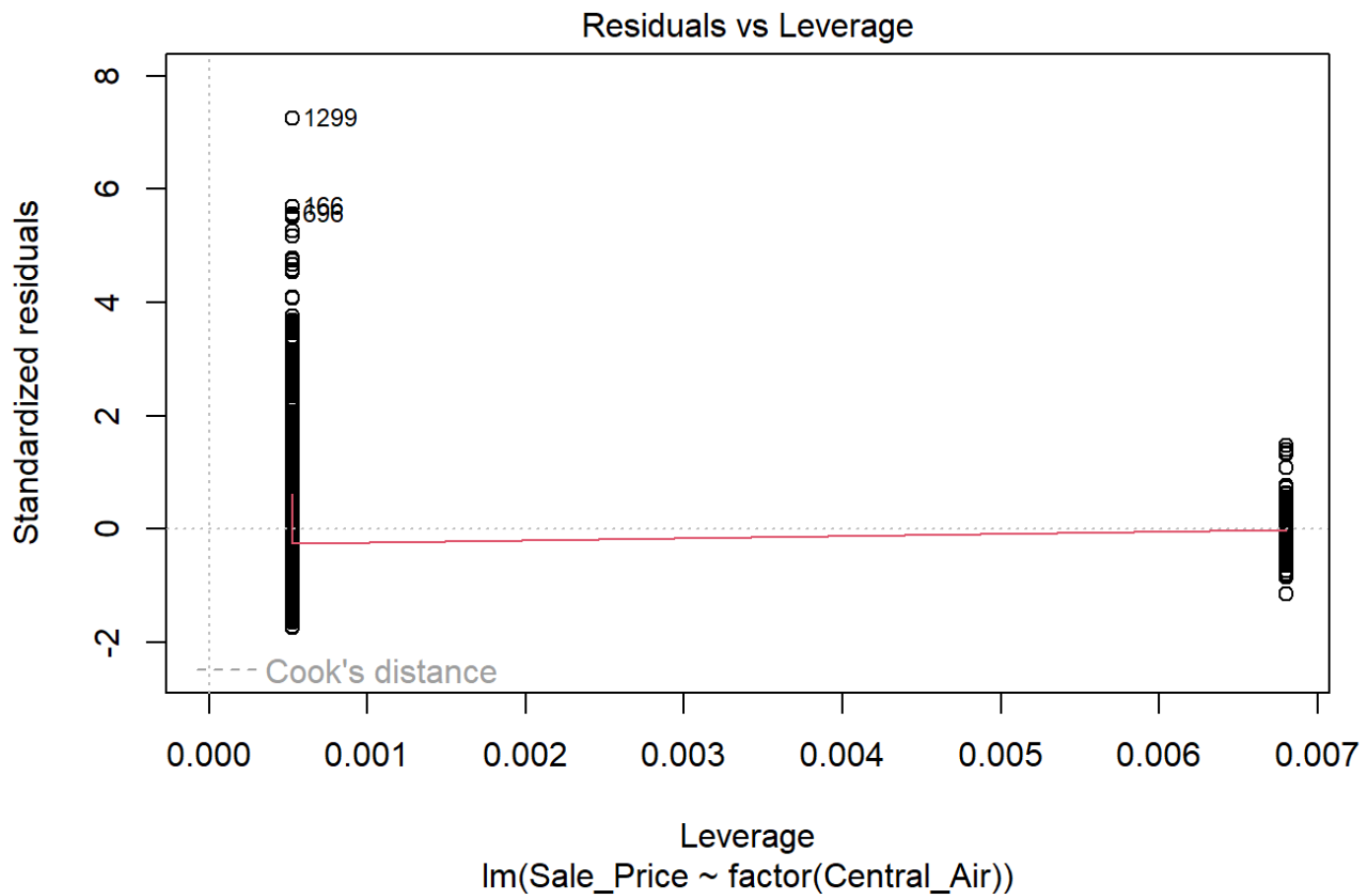
If we want to do this for a categorical variable with two levels (for example with Central Air), we would use the following code:

```
slr <- lm(Sale_Price ~ factor(Central_Air), data=train)
```

```
plot(slr)
```







The variance does NOT look constant and Normality is not a good assumption here. However, just for illustration purposes, here is the output:

```
summary(slr)
```

```
##
## Call:
## lm(formula = Sale_Price ~ factor(Central_Air), data = train)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -135003  -50003  -18003   28622  559997
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)      101065      6374   15.86  <2e-16 ***
## factor(Central_Air)Y    83938      6615   12.69  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 77270 on 2049 degrees of freedom
## Multiple R-squared:  0.07286,    Adjusted R-squared:  0.0724
## F-statistic:   161 on 1 and 2049 DF,  p-value: < 2.2e-16
```

We see that Central does indeed have a significant linear relationship with Sales Price. Looking at the above output, we see that X took on the value of 1 if the category was “Yes” and 0 if it was “No”. Therefore, we can interpret the coefficient as we expect the sales price of a home in Ames, Iowa to be on average \$83,938 higher if they have central air compared to those homes that do not have central air.

2.7.3 Python Code

Assumptions of Linear Regression

```
sp.stats.spearmanr(train['Gr_Liv_Area'], train['Sale_Price'])
```

```
## SignificanceResult(statistic=0.7160107413159085, pvalue=3.66e-322)
```

Test for Association

```
model_slr = smf.ols("Sale_Price ~ Gr_Liv_Area", data = train).fit()  
model_slr.summary()
```

OLS Regression Results						
Dep. Variable:	Sale_Price			R-squared:	0.488	
Model:	OLS			Adj. R-squared:	0.488	
Method:	Least Squares			F-statistic:	1952.	
Date:	Wed, 19 Jun 2024			Prob (F-statistic):	4.20e-300	
Time:	11:44:02			Log-Likelihood:	-25385.	
No. Observations:	2051			AIC:	5.077e+04	
Df Residuals:	2049			BIC:	5.078e+04	
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	1.405e+04	3942.503	3.563	0.000	6314.141	2.18e+04
Gr_Liv_Area	110.7259	2.506	44.185	0.000	105.811	115.640

Omnibus:	385.081	Durbin-Watson:	2.022
Prob(Omnibus):	0.000	Jarque-Bera (JB):	4372.360
Skew:	0.535	Prob(JB):	0.00
Kurtosis:	10.072	Cond. No.	4.89e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

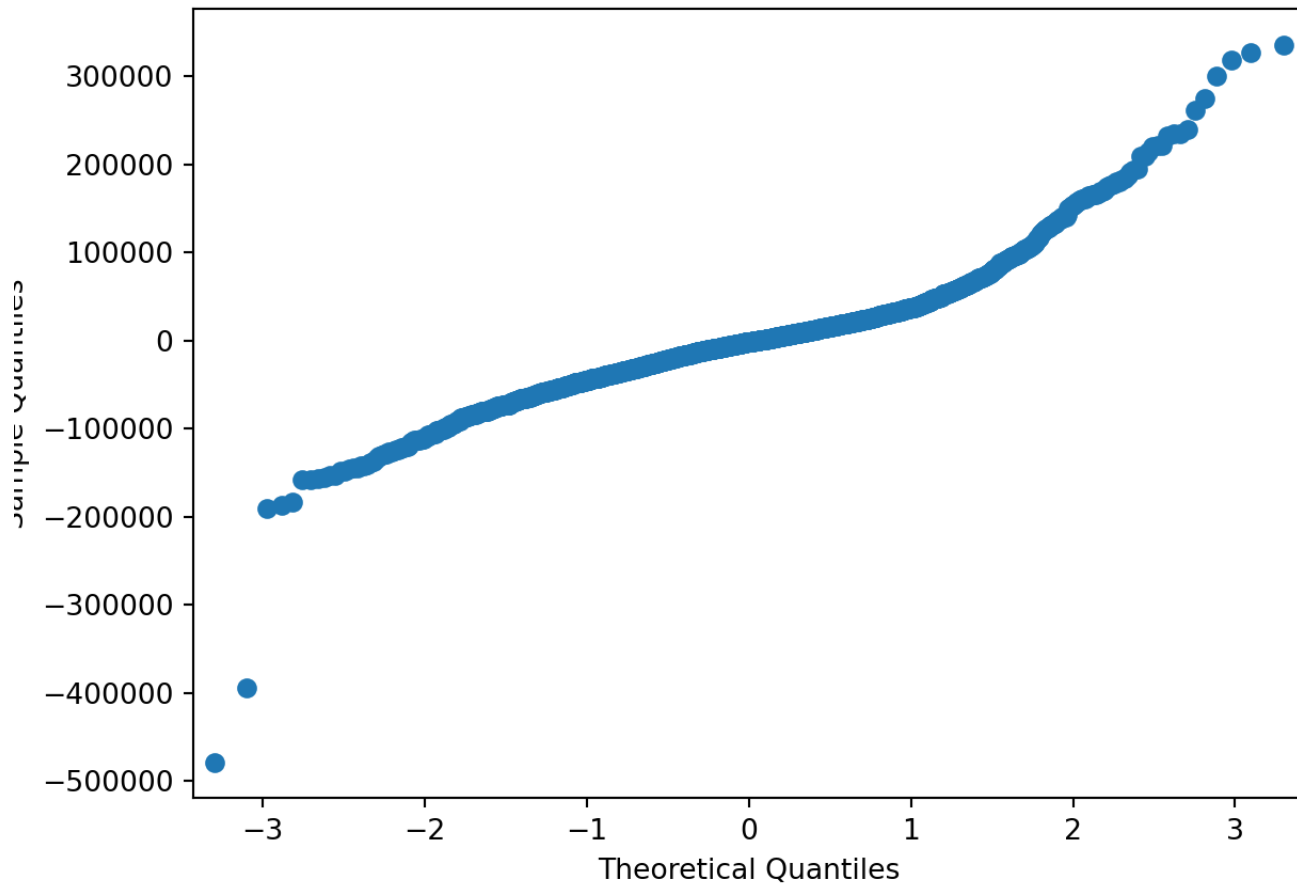
[2] The condition number is large, 4.89e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
train['pred_slr'] = model_slr.predict()
train['resid_slr'] = model_slr.resid

train[['Sale_Price', 'pred_anova', 'pred_slr']].head(n = 10)
```

```
##      Sale_Price      pred_anova      pred_slr
## 0         232600  228909.642647  178916.802008
## 1         166000  228909.642647  182792.210023
## 2         170000  142107.306719  200065.457178
## 3         252000  228909.642647  230293.639700
## 4         134000  142107.306719  139387.640249
## 5         164700  228909.642647  152453.301559
## 6         193500  142107.306719  199954.731235
## 7         118500  142107.306719  148688.619487
## 8          94000  142107.306719  111041.798764
## 9         111250  142107.306719  109713.087445
```

```
sm.api.qqplot(train['resid_slr'])  
plt.show()
```



```

import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(drop='first') # drop='first' to avoid dummy variable trap
central_air_encoded = encoder.fit_transform(train[['Central_Air']])

# Convert encoded variable to DataFrame
central_air_encoded_df = pd.DataFrame(central_air_encoded.toarray(), columns=encoder.get_feature_names_out())

# Concatenate the encoded variable with the original DataFrame
train_encoded = pd.concat([train[['Sale_Price']], central_air_encoded_df], axis=1)

# Define the feature matrix (X) and target variable (y)
X = train_encoded[['Central_Air_Y']] # Selecting the encoded column
y = train_encoded['Sale_Price']

# Initialize and fit the linear regression model
model = LinearRegression()
model.fit(X, y)

# Output the coefficients

```

▼ LinearRegression

LinearRegression()

```

print("Intercept:", model.intercept_)

```

```
## Intercept: 101065.34013605444
```

```
print("Coefficient for Central_Air_Y:", model.coef_[0])
```

```
## Coefficient for Central_Air_Y: 83938.00230092036
```

1. The n-dimensional “variable vectors” live in the vast *sample space* where the i^{th} axis represents the i^{th} observation in your dataset. In this space, a single point/vector is one possible set of sample values of n observations; this space can be difficult to grasp mentally ↩